LangChain: Complete Educational Guide

Building AI-Powered Applications with Language Models

Author: Technical Documentation
Version: 2.0 (Updated for LangChain 0.1+, LCEL-First)
Date: 2026
Target Audience: Intermediate Python developers building LLM applications

---

Table of Contents

Introduction & Architecture

Theory & Core Concepts

What is LangChain?

LangChain is a framework for developing applications powered by large language models (LLMs). It simplifies the complexity of building production-ready LLM applications by providing:

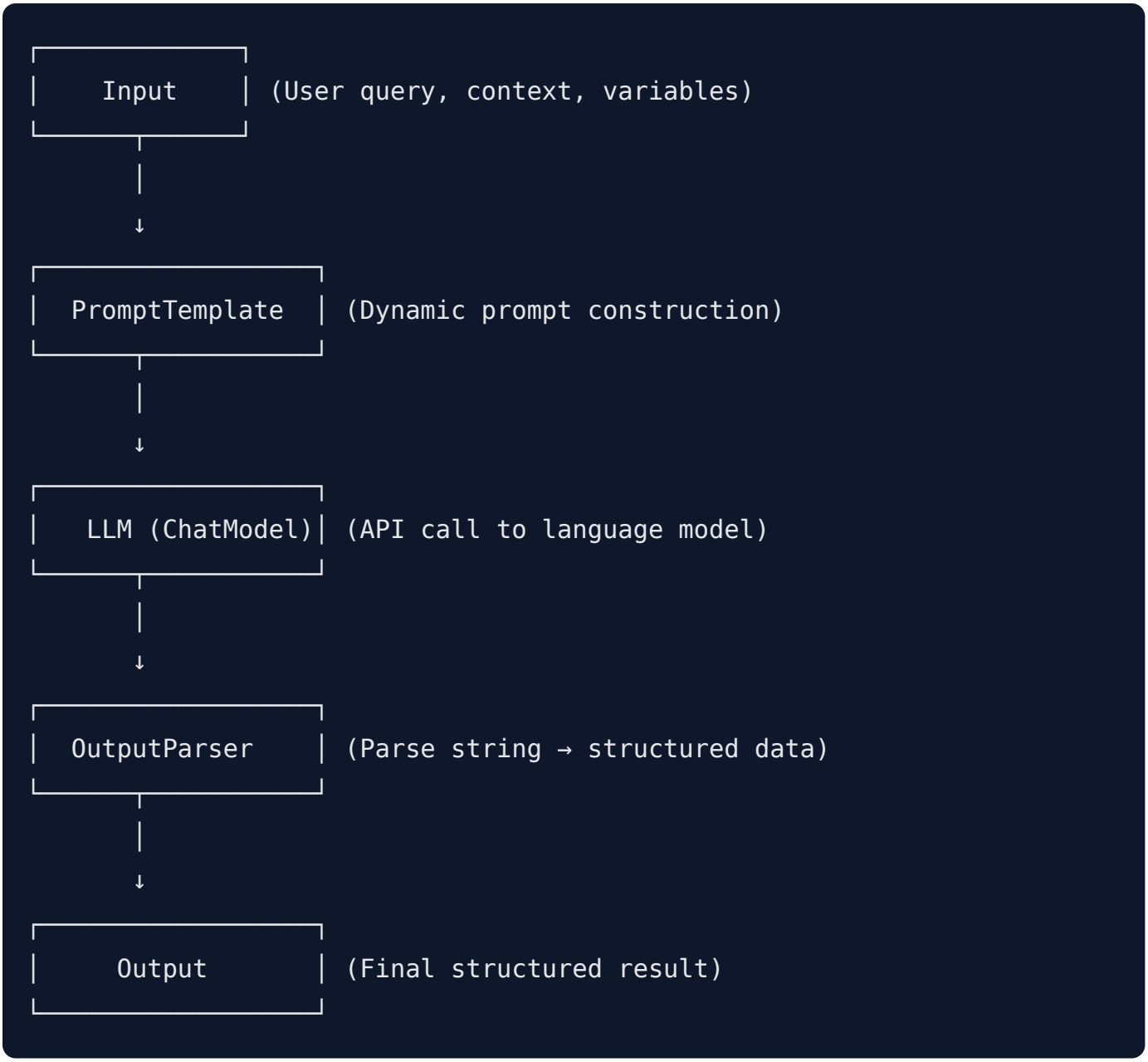
- **Abstractions** over LLM APIs (OpenAI, Anthropic, Ollama, etc.)
- **Composable components** that can be chained together using LCEL (LangChain Expression Language)
- **Memory management** for maintaining conversation context
- **Tool integration** for agents to interact with external APIs and databases
- **Prompt management** with templating and dynamic variable injection

The Problem LangChain Solves

Building LLM applications is complex because:

1. **API Fragmentation**: Different LLM providers have different interfaces
2. **Prompt Engineering Complexity**: Managing dynamic prompts at scale is error-prone
3. **Memory State**: Tracking conversation history manually is tedious
4. **Tool Integration**: Connecting LLMs to external tools requires orchestration
5. **Output Parsing**: LLMs return strings; extracting structured data is manual
6. **Composition**: Chaining multiple operations (LLM → Parser → Tool) was verbose

Core Architecture



Key Design Principles

1. **LCEL (LangChain Expression Language)**: Use `|` (pipe) operator to compose chains
2. **Runnables**: Core abstraction - anything with `.invoke()`, `.stream()`, `.batch()` methods
3. **Separation of Concerns**: Prompts, models, parsers are independent and reusable
4. **Flexibility**: Mix and match components, no vendor lock-in

---

Messages & Message Types

Theory & Core Concepts

What Are Messages?

Messages are the fundamental communication unit in LangChain. They represent discrete pieces of information exchanged in a conversation or LLM interaction.

Why Messages Exist:

- LLMs don't understand raw text requests uniformly
- Role-based communication (User, System, AI) improves model behavior
- Message history forms the foundation of stateful conversations
- Type safety ensures proper role assignment

Message Types in LangChain

LangChain provides three primary message types:

Message Type	Role	Purpose	Example
HumanMessage	User/Human	Query or instruction from the user	"What is the capital of France?"
SystemMessage	System	Context, role, or behavior instructions	"You are a helpful assistant"
AIMessage	Assistant	Response from the LLM	"The capital of France is Paris"

Message Architecture

Each message contains:

- **content** (str): The actual text
- **role** (implicit): Determined by message type
- **metadata** (optional): Additional context

Why Typed Messages Matter

1. **Semantic Clarity**: The model knows who's talking (user vs system vs assistant)
2. **Prompt Optimization**: Different roles trigger different model behaviors
3. **History Management**: Easy to replay conversation history accurately
4. **Tool Integration**: Agent systems rely on message roles for decision-making

---

Line-by-Line Code Breakdown

Basic Message Creation

```
from langchain_core.messages import HumanMessage, SystemMessage,
AIMessage

# Create a SystemMessage - sets the model's behavior/personality
system_msg = SystemMessage(content="You are a polite technical
assistant")
# Why we used it: Tells the model how to behave throughout the
conversation
# How it helps: Improves response quality by establishing context before
the user query

# Create a HumanMessage - represents user input
human_msg = HumanMessage(content="Explain quantum computing in simple
terms")
# Why we used it: Tells the model this is a user query, not system
context
# How it helps: Model treats it as a request, not instruction; maintains
role consistency

# Create an AIMessage - represents model's previous response
ai_msg = AIMessage(content="Quantum computing uses quantum bits
(qubits)...")
# Why we used it: Represents the assistant's perspective in multi-turn
conversations
# How it helps: Enables proper conversation history without the model
seeing its own output as user input
```

Message List for Conversation History

```
messages = [
    SystemMessage(content="You are a helpful assistant who explains
complex topics simply"),
    # First message in conversation
    HumanMessage(content="What is machine learning?"),
    # Model's response to first message
    AIMessage(content="Machine learning is a subset of AI where systems
learn from data..."),
    # Second human message continues conversation
    HumanMessage(content="Give me an example"),
]

# Why use a list: Preserves conversation flow; model can refer to
previous context
# How it helps: Enables multi-turn dialogue where model remembers
context; avoids rephrasing context repeatedly
```

Key Parameters Explained

```
HumanMessage(
    content="Your actual message text",
    # Why: The core content the LLM will process
    # How it helps: String content is what gets tokenized and sent to
the model
)

SystemMessage(
    content="You are an expert...",
    # Why: Sets system-level instructions
    # How it helps: Applied once at conversation start to "prime" the
model's behavior
)

AIMessage(
    content="Response text",
    # Why: Marks this as the assistant's output
    # How it helps: Prevents the model from treating its own previous
responses as user input
)
```

Complete, Ready-to-Run Code Block

```
"""
Complete Message Management Example
Demonstrates all message types and conversation flow
"""

from langchain_core.messages import HumanMessage, SystemMessage,
AIMessage
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()
if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not found in environment")

# Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# Create a conversation message list
messages = [
    SystemMessage(
        content="You are a world-class technical writer. Explain
concepts clearly and concisely."
    ),
    HumanMessage(content="What is an API?"),
    AIMessage(content="An API (Application Programming Interface) is a
set of protocols..."),
    HumanMessage(content="Give me a real-world example"),
]

# Invoke the LLM with messages
response = llm.invoke(messages)

# Extract response content
assistant_response = response.content
print("Assistant:", assistant_response)
```

```
# Add the response to conversation history
messages.append(AIMessage(content=assistant_response))

# Continue conversation
new_human_message = HumanMessage(content="How does this relate to REST
APIs?")
messages.append(new_human_message)

# Get next response
response2 = llm.invoke(messages)
print("\nAssistant:", response2.content)

# Complete conversation history at this point
messages.append(AIMessage(content=response2.content))

print("\n--- Complete Conversation History ---")
for i, msg in enumerate(messages, 1):
    msg_type = type(msg).__name__
    print(f"{i}. [{msg_type}] {msg.content[:60]}...")
```

Detailed Data Workflow Explanation

Input Flow

```
Step 1: Message Creation
├─ SystemMessage(content="You are...")
├─ HumanMessage(content="User query")
├─ AIMessage(content="Previous response")
└─ List combined → messages = [sys, human, ai]

Step 2: List to LLM
├─ messages list passed to llm.invoke()
└─ Internal conversion to API format

Step 3: API Transformation
├─ Messages serialized to JSON
├─ Roles mapped to API format (e.g., "system", "user", "assistant")
└─ Sent to OpenAI/other provider API

Step 4: Response Generation
├─ LLM processes message context
├─ Generates response text
└─ Returns as BaseMessage object

Step 5: Output Extraction
├─ response.content extracted as string
├─ Can append back to messages as AIMessage(content=response.content)
└─ Ready for next iteration
```

State Management

```
Initial State:
  messages = [SystemMessage, HumanMessage]

After First Invoke:
  response = llm.invoke(messages)
  messages.append(AIMessage(response.content))
  messages = [SystemMessage, HumanMessage, AIMessage]

After Second Human Input:
  messages.append(HumanMessage(new_query))
  response2 = llm.invoke(messages)
  messages.append(AIMessage(response2.content))
  messages = [SystemMessage, HumanMessage, AIMessage, HumanMessage,
AIMessage]
```

Memory State Updates

- **System Message**: Stays constant throughout conversation (role definition)
- **Human/AI Messages**: Append in alternating pattern to preserve turn-taking
- **Growth**: Each turn adds 1-2 messages; cumulative cost grows with conversation length
- **Truncation Strategy**: For long conversations, implement message window (keep only recent N messages) to manage token cost

---

LLM Integration

Theory & Core Concepts

What is an LLM in LangChain?

An LLM is the core model that generates responses. LangChain abstracts different LLM providers (OpenAI, Anthropic, Ollama, Hugging Face, etc.) behind a unified `ChatModel` interface.

Why LLM Integration Matters

1. **Provider Abstraction**: Same code works with different LLM providers
2. **Parameter Control**: Temperature, max_tokens, stop sequences tune model behavior

3. **Error Handling**: Built-in retries and fallback mechanisms
4. **Streaming Support**: Real-time token streaming for better UX
5. **Batch Processing**: Efficient processing of multiple requests

Core LLM Parameters

Parameter	Range	Purpose	Default
temperature	0.0 - 2.0	Randomness/creativity (0=deterministic, 2=very random)	1.0
max_tokens	1 - model limit	Maximum response length in tokens	Unlimited
top_p	0.0 - 1.0	Nucleus sampling (lower = more focused)	1.0
frequency_penalty	-2.0 to 2.0	Penalize repeated tokens	0.0
presence_penalty	-2.0 to 2.0	Penalize new topics	0.0

API Keys & Environment Setup

Critical: Never hardcode API keys. Always use environment variables:

- `OPENAI_API_KEY` for OpenAI
- `ANTHROPIC_API_KEY` for Anthropic
- `OLLAMA_BASE_URL` for local Ollama

Line-by-Line Code Breakdown

Basic LLM Initialization

```
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()
# Why: Keeps API keys out of source code; enables secure credential
management
# How it helps: Prevents accidental key leaks in version control;
enables per-environment configuration

# Check if API key is loaded
if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY environment variable not set")
# Why: Fail fast with clear error instead of cryptic API error later
# How it helps: Immediate feedback during development; easier debugging

# Initialize ChatOpenAI model with parameters
llm = ChatOpenAI(
    model="gpt-3.5-turbo",  # Specify which model to use
    # Why: Older models are cheaper; newer models are smarter; choose
based on cost/performance tradeoff

    temperature=0.7,  # Controls randomness (0=deterministic,
1=balanced, 2=very random)
    # Why: 0 for factual tasks, 0.7 for creative tasks, higher for
brainstorming
    # How it helps: Lower temperature gives consistent results; higher
gives variety

    max_tokens=500,  # Limit response length
    # Why: Prevent runaway responses; save tokens/cost
    # How it helps: Enforces conciseness; stops model from over-
explaining
)
```

Invoking the LLM

```
from langchain_core.messages import HumanMessage, SystemMessage

# Create message list
messages = [
    SystemMessage(content="You are a helpful assistant"),
    HumanMessage(content="What is Python?")
]

# Method 1: invoke() - synchronous, single response
response = llm.invoke(messages)
# Why: Standard method for single queries
# How it helps: Blocks until response is complete; simple synchronous
flow

content = response.content
# Why: Extract string from BaseMessage object
# How it helps: Get actual text from the response object

print(content)  # Output: "Python is a programming language..."

# Method 2: stream() - streaming responses token-by-token
print("Streaming response:")
for chunk in llm.stream(messages):
    print(chunk.content, end="", flush=True)
# Why: Show responses in real-time as tokens arrive
# How it helps: Better UX; don't wait for full response; can cancel mid-
stream

# Method 3: batch() - process multiple queries efficiently
queries = [
    HumanMessage(content="What is Python?"),
    HumanMessage(content="What is JavaScript?"),
    HumanMessage(content="What is Go?"),
]
responses = llm.batch(queries)
# Why: Parallel processing of multiple requests
```

```
# How it helps: More efficient than calling invoke() three times; better throughput
```

Temperature Impact

```
# Temperature = 0 (Deterministic)
llm_deterministic = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
# Use for: Factual questions, customer service, code generation
# Always gives same response to same input

# Temperature = 0.7 (Balanced)
llm_balanced = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
# Use for: General conversations, Q&A
# Balanced creativity and consistency

# Temperature = 1.5 (Creative)
llm_creative = ChatOpenAI(model="gpt-3.5-turbo", temperature=1.5)
# Use for: Brainstorming, creative writing, idea generation
# High variation; responses differ each call
```

Complete, Ready-to-Run Code Block

```
"""
Complete LLM Integration Example
Demonstrates initialization, invocation modes, and parameter tuning
"""

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage
import os
from dotenv import load_dotenv

# ===== SETUP =====
load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("Please set OPENAI_API_KEY environment variable")

# ===== INITIALIZATION =====

# Create LLM instance with tuned parameters
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0.7,
    max_tokens=1000,
    timeout=30  # 30 second timeout for API calls
)

# ===== SINGLE INVOKE =====

def single_query_example():
    """Simple single query with response"""
    messages = [
        SystemMessage(content="You are a Python expert"),
        HumanMessage(content="Explain decorators in Python")
    ]

    response = llm.invoke(messages)
    print("=" * 50)
```

```
    print("SINGLE INVOKE EXAMPLE")
    print("=" * 50)
    print(response.content)
    return response

# ===== STREAMING =====

def streaming_example():
    """Stream response tokens as they arrive"""
    messages = [
        HumanMessage(content="Write a haiku about programming")
    ]

    print("\n" + "=" * 50)
    print("STREAMING EXAMPLE (tokens arrive in real-time)")
    print("=" * 50)

    for chunk in llm.stream(messages):
        print(chunk.content, end="", flush=True)
    print()  # Newline after streaming

# ===== BATCH PROCESSING =====

def batch_example():
    """Process multiple queries efficiently"""
    queries = [
        HumanMessage(content="What is list comprehension?"),
        HumanMessage(content="Explain lambda functions"),
        HumanMessage(content="What are generators?"),
    ]

    print("\n" + "=" * 50)
    print("BATCH PROCESSING EXAMPLE")
    print("=" * 50)

    responses = llm.batch(queries)
```

```
    for i, (query, response) in enumerate(zip(queries, responses), 1):
        print(f"\nQuery {i}: {query.content}")
        print(f"Response: {response.content[:200]}...")

# ===== TEMPERATURE COMPARISON =====

def temperature_comparison():
    """Compare same query with different temperatures"""
    message = HumanMessage(content="Complete this: AI is...")

    print("\n" + "=" * 50)
    print("TEMPERATURE COMPARISON")
    print("=" * 50)

    temperatures = [0, 0.7, 1.5]

    for temp in temperatures:
        llm_temp = ChatOpenAI(
            model="gpt-3.5-turbo",
            temperature=temp
        )
        response = llm_temp.invoke([message])
        print(f"\nTemperature {temp}:")
        print(f"  {response.content[:100]}...")

# ===== ERROR HANDLING =====

def error_handling_example():
    """Demonstrate proper error handling"""
    print("\n" + "=" * 50)
    print("ERROR HANDLING EXAMPLE")
    print("=" * 50)

    try:
        # Attempt to call LLM
        llm_risky = ChatOpenAI(
            model="gpt-3.5-turbo",
```

```
            timeout=0.001  # Unreasonably short timeout to trigger error
        )
        response = llm_risky.invoke([
            HumanMessage(content="Hello")
        ])
    except Exception as e:
        print(f"Error occurred (expected): {type(e).__name__}")
        print(f"Error message: {str(e)[:100]}...")
        print("✓ Error handled gracefully")

# ===== MULTI-TURN CONVERSATION =====

def multi_turn_conversation():
    """Maintain conversation state across multiple turns"""
    print("\n" + "=" * 50)
    print("MULTI-TURN CONVERSATION")
    print("=" * 50)

    messages = [
        SystemMessage(content="You are a helpful math tutor")
    ]

    # Turn 1
    messages.append(HumanMessage(content="What is 2+2?"))
    response1 = llm.invoke(messages)
    messages.append(response1)
    print(f"\nTurn 1 - Assistant: {response1.content}")

    # Turn 2
    messages.append(HumanMessage(content="What about 3+4?"))
    response2 = llm.invoke(messages)
    messages.append(response2)
    print(f"Turn 2 - Assistant: {response2.content}")

    # Turn 3
    messages.append(HumanMessage(content="Add the two results"))
    response3 = llm.invoke(messages)
```

```
    messages.append(response3)
    print(f"Turn 3 - Assistant: {response3.content}")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    # Run all examples
    single_query_example()
    streaming_example()
    batch_example()
    temperature_comparison()
    error_handling_example()
    multi_turn_conversation()

    print("\n" + "=" * 50)
    print("✓ All LLM integration examples completed")
    print("=" * 50)
```

Detailed Data Workflow Explanation

Single Invoke Workflow

```
Input:
  messages = [SystemMessage(...), HumanMessage(...)]
  |
  └─→ llm.invoke(messages)

Processing:
  1. Serialize messages to API format
     {"messages": [{"role": "system", "content": "..."}, {"role":
"user", "content": "..."}]}

  2. Network request to OpenAI API
     POST https://api.openai.com/v1/chat/completions

  3. Model processes input
     - Tokenizes messages
     - Generates response token-by-token (hidden from user)
     - Stops at max_tokens or natural endpoint

  4. Receive full response
     {"choices": [{"message": {"role": "assistant", "content": "..."}}]}

Output:
  AIMessage(content="Response text")
  |
  └─→ response.content = "Response text" (string)
```

Streaming Workflow

```
Input:
  messages = [HumanMessage(...)]
  │
  └─→ llm.stream(messages)

Processing:
  1. Same serialization as invoke()
  2. API request sent
  3. Model processes AND streams response:

     Chunk 1: {"content": "Hello"}
     Chunk 2: {"content": " there"}
     Chunk 3: {"content": ","}
     Chunk 4: {"content": " how"}
     Chunk 5: {"content": " are"}
     Chunk 6: {"content": " you"}
     (END OF RESPONSE)

Output:
  Iterator yields each chunk:
  for chunk in stream:
    print(chunk.content, end="", flush=True)

  Displays: "Hello there, how are you" (token by token)

  Benefit: User sees response immediately; doesn't wait for full
response
```

Batch Workflow

```
Input:
  queries = [HumanMessage(q1), HumanMessage(q2), HumanMessage(q3)]
  │
  └─→ llm.batch(queries)

Processing:
  1. Parallel processing (multiple concurrent API calls)

     Request 1: POST .../completions (q1)
     Request 2: POST .../completions (q2)
     Request 3: POST .../completions (q3)

     All run simultaneously

  2. Wait for all responses

  3. Collect responses in order

Output:
  [AIMessage(response1), AIMessage(response2), AIMessage(response3)]

  Benefit: Process 3 queries in ~(time of 1 query) instead of 3x that
```

Prompts & PromptTemplate

Theory & Core Concepts

What is a Prompt?

A prompt is the instruction(s) you send to an LLM. In LangChain, prompts are more than strings—they're templated, dynamic, composable objects.

Types of Prompts

1. **Static Prompts**: Fixed text, no variables

```
"What is Python?"
```

2. **Template Prompts**: Contain variables that get filled in

```
"What is {language}?"  (where {language} = "Python")
```

3. **Message Prompts**: Structured with system/user/assistant roles

```
System: "You are a helpful assistant"
User: "What is {topic}?"
```

Why PromptTemplate Matters

1. **Reusability**: Define once, use with different variables
2. **Consistency**: Ensure all invocations use same prompt structure
3. **Validation**: Catch missing variables before API call
4. **Composition**: Combine templates into larger chains
5. **Documentation**: Template itself documents what variables are needed

Prompt Template Architecture

```
PromptTemplate
├─ template: str (contains {variable} placeholders)
├─ input_variables: List[str] (["variable"])
└─ validate_template: bool (check for errors)

ChatPromptTemplate
├─ messages: List[Tuple[str, str]] (role, template pairs)
├─ input_variables: List[str] (extracted from all templates)
└─ from_messages(): class method (convenient construction)
```

Line-by-Line Code Breakdown

Basic PromptTemplate

```
from langchain_core.prompts import PromptTemplate

# Create a simple template with one variable
template = PromptTemplate(
    template="Tell me a fun fact about {topic}",
    # Why: Defines the template with {topic} as a variable placeholder
    # How it helps: The {topic} will be replaced with actual values at
runtime

    input_variables=["topic"]
    # Why: Explicitly declares which variables this template expects
    # How it helps: Validation - ensures template won't fail with
missing variables
)

# Format the template with actual values
formatted_prompt = template.invoke({"topic": "Python"})
# Why: invoke() replaces {topic} with "Python"
# How it helps: Produces: "Tell me a fun fact about Python"

print(formatted_prompt)  # Output: "Tell me a fun fact about Python"
```

ChatPromptTemplate with System & User Messages

```
from langchain_core.prompts import ChatPromptTemplate

# Create structured prompt with roles
prompt_template = ChatPromptTemplate.from_messages([
    (
        "system",  # Role identifier
        "You are a helpful assistant specializing in {specialty}"  # Template text
        # Why system role: Sets model's behavior/persona for entire conversation
        # How it helps: Model knows its context before seeing user query
    ),
    (
        "user",  # Role identifier
        "Answer this {question} in {word_count} words"  # Template text
        # Why user role: Marks this as the user's actual request
        # How it helps: Model knows this is the query, not system instruction
    )
])

# Input variables are automatically extracted from all templates
input_variables = prompt_template.input_variables
# Output: ["specialty", "question", "word_count"]
# Why auto-extraction: Don't manually specify; framework infers from {variable} patterns
# How it helps: Less error-prone; maintains consistency

# Format with values
formatted = prompt_template.invoke({
    "specialty": "Python programming",
    "question": "How do decorators work?",
    "word_count": "50"
})
# Why invoke(): Replaces all variables with provided values
# How it helps: Produces a list of Message objects ready for LLM
```

Dynamic Prompt Construction

```
from langchain_core.prompts import PromptTemplate

# Create template
template = PromptTemplate(
    template="Answer this question:\n{question}\n\nContext: {context}",
    input_variables=["question", "context"]
)

# Later, you have dynamic values
user_question = "What is machine learning?"
retrieved_context = "ML is a subset of AI..."

# Invoke with dynamic values
prompt = template.invoke({
    "question": user_question,
    "context": retrieved_context
})
# Why: Supports runtime data injection
# How it helps: Same template works with different contexts; essential
for RAG
```

Partial Templates (Pre-fill Some Variables)

```
from langchain_core.prompts import PromptTemplate

# Original template requires 3 variables
template = PromptTemplate(
    template="Answer as {persona}:\n{question}\n\nFormat: {format}",
    input_variables=["persona", "question", "format"]
)

# Pre-fill one variable
partial_template = template.partial(persona="a Python expert")
# Why: Reduce number of variables needed later
# How it helps: Now only needs ["question", "format"] when invoked

# Now invoke with fewer variables
prompt = partial_template.invoke({
    "question": "Explain decorators",
    "format": "bullet points"
})
# Automatically includes: persona="a Python expert"
```

Complete, Ready-to-Run Code Block

```
"""
Complete PromptTemplate Example
Demonstrates static, dynamic, partial, and chat prompts
"""

from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_openai import ChatOpenAI
import os
from dotenv import load_dotenv

load_dotenv()

# ===== SETUP =====

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# ===== EXAMPLE 1: BASIC PROMPT TEMPLATE =====

def example_basic_template():
    """Simple template with one variable"""
    print("\n" + "=" * 60)
    print("EXAMPLE 1: BASIC PROMPT TEMPLATE")
    print("=" * 60)

    # Define template
    prompt_template = PromptTemplate(
        template="Tell me a fun fact about {topic}",
        input_variables=["topic"]
    )

    # Format and display
    for topic in ["Python", "Space", "Dinosaurs"]:
        formatted = prompt_template.invoke({"topic": topic})
        print(f"\nTemplate with topic='{topic}':")
```

```
        print(f"  {formatted}")

        # Pass to LLM
        response = llm.invoke([formatted])
        print(f"  Response: {response.content[:100]}...")

# ===== EXAMPLE 2: MULTI-VARIABLE TEMPLATE =====

def example_multi_variable():
    """Template with multiple variables"""
    print("\n" + "=" * 60)
    print("EXAMPLE 2: MULTI-VARIABLE TEMPLATE")
    print("=" * 60)

    # Define multi-variable template
    template = PromptTemplate(
        template="""
        Answer the following {question_type}:
        Question: {question}
        Level: {difficulty_level}
        Format your answer in {format_type} format.
        """,
        input_variables=["question_type", "question",
"difficulty_level", "format_type"]
    )

    # Use with different values
    response = llm.invoke([
        template.invoke({
            "question_type": "technical",
            "question": "How does a hash map work?",
            "difficulty_level": "intermediate",
            "format_type": "step-by-step"
        })
    ])

    print("Question: How does a hash map work?")
```

```
    print(f"Response: {response.content}")

# ===== EXAMPLE 3: CHAT PROMPT TEMPLATE =====

def example_chat_template():
    """Structured template with system and user roles"""
    print("\n" + "=" * 60)
    print("EXAMPLE 3: CHAT PROMPT TEMPLATE")
    print("=" * 60)

    # Define chat template
    chat_template = ChatPromptTemplate.from_messages([
        (
            "system",
            "You are a {profession} expert with {years} years of
experience"
        ),
        (
            "user",
            "Explain {concept} to someone with {skill_level} skills"
        )
    ])

    # Format into messages
    messages = chat_template.invoke({
        "profession": "software engineer",
        "years": "10",
        "concept": "REST APIs",
        "skill_level": "beginner"
    })

    print(f"\nGenerated messages ({len(messages)} total):")
    for i, msg in enumerate(messages, 1):
        print(f"  {i}. [{msg.type}] {msg.content[:60]}...")

    # Send to LLM
    response = llm.invoke(messages)
```

```
    print(f"\nLLM Response:\n{response.content}")

# ===== EXAMPLE 4: PARTIAL TEMPLATE =====

def example_partial_template():
    """Pre-fill some variables, fill others later"""
    print("\n" + "=" * 60)
    print("EXAMPLE 4: PARTIAL TEMPLATE (Pre-filled Variables)")
    print("=" * 60)

    # Full template requires 3 variables
    template = PromptTemplate(
        template="Answer as {persona}:\nQuestion: {question}\nFormat:
{format}",
        input_variables=["persona", "question", "format"]
    )

    # Pre-fill one variable
    expert_template = template.partial(persona="a Python expert with 15
years experience")

    print("Original template variables:", template.input_variables)
    print("After partial() variables:", expert_template.input_variables)

    # Now only need question and format
    formatted = expert_template.invoke({
        "question": "What are decorators and why use them?",
        "format": "concise bullet points"
    })

    print(f"\nFormatted prompt:\n{formatted}")

    response = llm.invoke([formatted])
    print(f"\nResponse: {response.content[:200]}...")

# ===== EXAMPLE 5: DYNAMIC CONTEXT (FOR RAG) =====
```

```
def example_dynamic_context():
    """Template with dynamic context (common in RAG)"""
    print("\n" + "=" * 60)
    print("EXAMPLE 5: DYNAMIC CONTEXT (RAG Pattern)")
    print("=" * 60)

    # RAG-style template
    rag_template = PromptTemplate(
        template="""
        Context information:
        {context}

        Based on the context above, answer: {question}
        """,
        input_variables=["context", "question"]
    )

    # Simulate retrieved context
    retrieved_context = """
    Python is a high-level programming language known for:
    - Simple, readable syntax
    - Powerful libraries (NumPy, Pandas, Django)
    - Versatility (web, AI, data science, automation)
    """

    formatted = rag_template.invoke({
        "context": retrieved_context,
        "question": "Why is Python popular for data science?"
    })

    print(f"Prompt:\n{formatted}")

    response = llm.invoke([formatted])
    print(f"\nResponse: {response.content}")

# ===== EXAMPLE 6: REUSABLE PROMPT LIBRARY =====
```

```
def example_prompt_library():
    """Build reusable prompt library"""
    print("\n" + "=" * 60)
    print("EXAMPLE 6: REUSABLE PROMPT LIBRARY")
    print("=" * 60)

    # Define library of prompts
    prompts = {
        "summarize": PromptTemplate(
            template="Summarize this in {word_count} words:\n{text}",
            input_variables=["word_count", "text"]
        ),
        "translate": PromptTemplate(
            template="Translate to {language}:\n{text}",
            input_variables=["language", "text"]
        ),
        "code_review": PromptTemplate(
            template="Review this {language} code for issues:\n{code}",
            input_variables=["language", "code"]
        )
    }

    # Use from library
    summarize_prompt = prompts["summarize"].invoke({
        "word_count": "50",
        "text": "Python is a programming language. It is versatile. Many
use it for AI."
    })

    print("Summarize task:")
    print(summarize_prompt)

    translate_prompt = prompts["translate"].invoke({
        "language": "Spanish",
        "text": "Hello, how are you?"
    })
```

```
    print(f"\nTranslate task:")
    print(translate_prompt)

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_template()
    example_multi_variable()
    example_chat_template()
    example_partial_template()
    example_dynamic_context()
    example_prompt_library()

    print("\n" + "=" * 60)
    print("✓ All prompt template examples completed")
    print("=" * 60)
```

Detailed Data Workflow Explanation

Template Invocation Flow

```
Step 1: Define Template
  template = "Explain {concept} in {difficulty} terms"
  input_variables = ["concept", "difficulty"]

Step 2: Prepare Input Data
  input_dict = {
    "concept": "machine learning",
    "difficulty": "simple"
  }

Step 3: Invoke Template
  template.invoke(input_dict)

Step 4: Variable Replacement
  "Explain {concept} in {difficulty} terms"
       ↓ (concept → machine learning)
  "Explain machine learning in {difficulty} terms"
       ↓ (difficulty → simple)
  "Explain machine learning in simple terms"

Step 5: Return to Caller
  formatted_prompt = "Explain machine learning in simple terms"
```

Chat Template Message Generation

```
Step 1: Define Structure
  messages = [
    ("system", "You are a {role}"),
    ("user", "Explain {topic}")
  ]

Step 2: Input Values
  values = {"role": "Python expert", "topic": "decorators"}

Step 3: Replace in Each Template
  System: "You are a {role}" + role="Python expert"
          → "You are a Python expert"

  User:   "Explain {topic}" + topic="decorators"
          → "Explain decorators"

Step 4: Create Message Objects
  [
    SystemMessage(content="You are a Python expert"),
    HumanMessage(content="Explain decorators")
  ]

Step 5: Return Message List Ready for LLM
  messages → pass to llm.invoke(messages)
```

Variable Extraction

```
Template: "Answer as {persona}:\n{question}\nFormat: {format}"

Regex scan for {variable} patterns:
  Found: persona
  Found: question
  Found: format

Extracted input_variables = ["persona", "question", "format"]

When invoke() called:
  - Check that all variables in input_variables are provided
  - Throw error if missing
  - Replace all occurrences of each variable
  - Return formatted string
```

Output Parsing

Theory & Core Concepts

What is Output Parsing?

LLMs return plain text strings. Output parsing converts these strings into structured data (JSON, dictionaries, Python objects, etc.).

Why Output Parsing Matters

1. **Type Safety**: Get dictionaries/objects, not strings
2. **Programmatic Access**: Access response fields like `response.joke_setup` instead of parsing manually
3. **Validation**: Ensure LLM output matches expected schema
4. **Error Handling**: Built-in retry logic if parsing fails
5. **Documentation**: Parser schema serves as contract between LLM and application

Types of Parsers

Parser	Output Type	Use Case

StrOutputParser	String	When you want raw LLM text
JSONOutputParser	Dictionary	When LLM returns JSON
PydanticOutputParser	Pydantic Model	Structured data with validation
CommaSeparatedListOutputParser	List	When LLM returns comma-separated values

Output Parsing Architecture

```
LLM Response (string)
    ↓
Parser
    ├─ Validate format
    ├─ Extract structure
    └─ Convert to target type
    ↓
Structured Output (dict, object, list, etc.)
```

Line-by-Line Code Breakdown

StrOutputParser (String Output)

```
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo")

# Create parser that extracts string content
parser = StrOutputParser()
# Why: Simple parser for text-only responses
# How it helps: Extracts .content from LLM response automatically

# Use in chain
response = llm.invoke([HumanMessage(content="Hello")])
# response is AIMessage object

output = parser.invoke(response)
# Why: parser.invoke() extracts the string
# How it helps: Now output is pure string, not BaseMessage object
```

JSONOutputParser (Dictionary Output)

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

# Create parser that converts JSON string to dict
parser = JsonOutputParser()
# Why: Converts JSON string to Python dictionary
# How it helps: Type-safe access to JSON fields

# Create prompt that asks for JSON
prompt_template = PromptTemplate(
    template="""Generate a joke in JSON format with fields 'setup' and
'punchline'

    {format_instructions}

    Topic: {topic}""",
    input_variables=["topic"],
    # Why: format_instructions will contain the format specification
)

# Add format instructions from parser
prompt = prompt_template.partial(
    format_instructions=parser.get_format_instructions()
    # Why: Tells LLM exactly how to format output
    # How it helps: LLM knows expected JSON structure
)

# Invoke chain
response = llm.invoke(prompt.invoke({"topic": "Python"}))
# response.content = '{"setup": "Why do...", "punchline": "Because..."}'

parsed = parser.invoke(response)
# Why: Converts JSON string to dict
# How it helps: parsed = {"setup": "...", "punchline": "..."}
# Now access as: parsed["setup"], parsed["punchline"]
```

PydanticOutputParser (Structured Objects)

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

# Define data schema using Pydantic
class Joke(BaseModel):
    setup: str = Field(description="The setup of the joke")
    punchline: str = Field(description="The punchline of the joke")
# Why Pydantic: Provides validation, documentation, type hints
# How it helps: Automatically validates output matches schema

# Create parser from schema
parser = PydanticOutputParser(pydantic_object=Joke)
# Why: Parser knows the expected structure
# How it helps: Can validate and create instances

# Get format instructions (tells LLM expected format)
format_instructions = parser.get_format_instructions()
# Output: "You must output a valid JSON..."

# Create prompt with format instructions
prompt_template = PromptTemplate(
    template="""Generate a {type_} joke

    {format_instructions}""",
    input_variables=["type_"],
    partial_variables={"format_instructions": format_instructions}
    # Why: Include format instructions in every invocation
    # How it helps: Guides LLM output format
)

# Chain: prompt → LLM → parser
response = llm.invoke(prompt_template.invoke({"type_": "dad"}))
parsed_joke = parser.invoke(response)
# parsed_joke is Joke object
```

```
# Access: parsed_joke.setup, parsed_joke.punchline
# Type-safe: IDE knows these attributes exist
```

Error Handling in Parsing

```
from langchain_core.output_parsers import PydanticOutputParser,
OutputParserException
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int

parser = PydanticOutputParser(pydantic_object=Person)

# Valid JSON
valid_json = '{"name": "Alice", "age": 30}'
try:
    result = parser.invoke(valid_json)
    print(result)  # Person(name='Alice', age=30)
except OutputParserException as e:
    # Why: Catch parsing errors
    # How it helps: Handle malformed output gracefully
    print(f"Parsing error: {e}")

# Invalid JSON (missing required field)
invalid_json = '{"name": "Bob"}'  # Missing age
try:
    result = parser.invoke(invalid_json)
except OutputParserException as e:
    print(f"Validation error: {e}")
    # Output: "Parsing error: Invalid JSON output: Field 'age' required"
```

Complete, Ready-to-Run Code Block

```
"""
Complete Output Parsing Example
Demonstrates all parser types and error handling
"""

from langchain_core.output_parsers import (
    StrOutputParser,
    JsonOutputParser,
    PydanticOutputParser,
    CommaSeparatedListOutputParser,
    OutputParserException
)
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
import json
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# ===== EXAMPLE 1: STR OUTPUT PARSER =====

def example_str_parser():
    """Simple string extraction from LLM response"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: STRING OUTPUT PARSER")
    print("=" * 70)

    parser = StrOutputParser()

    prompt = PromptTemplate(
```

```
        template="Write a one-line joke about {topic}",
        input_variables=["topic"]
    )

    # Create chain: prompt → LLM → parser
    chain = prompt | llm | parser

    result = chain.invoke({"topic": "Python"})
    print(f"Result type: {type(result)}")
    print(f"Result: {result}")

# ===== EXAMPLE 2: JSON OUTPUT PARSER =====

def example_json_parser():
    """Parse JSON output into dictionaries"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: JSON OUTPUT PARSER")
    print("=" * 70)

    json_parser = JsonOutputParser()

    prompt = PromptTemplate(
        template="""Generate a person profile in JSON format.

        {format_instructions}

        Profile for a {profession}""",
        input_variables=["profession"],
        partial_variables={
            "format_instructions": json_parser.get_format_instructions()
        }
    )

    chain = prompt | llm | json_parser

    result = chain.invoke({"profession": "software engineer"})
    print(f"Result type: {type(result)}")
```

```
    print(f"Result: {result}")

    # Access as dictionary
    if isinstance(result, dict):
        for key, value in result.items():
            print(f"  {key}: {value}")

# ===== EXAMPLE 3: PYDANTIC OUTPUT PARSER =====

def example_pydantic_parser():
    """Parse into strongly-typed Pydantic objects"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: PYDANTIC OUTPUT PARSER")
    print("=" * 70)

    # Define schema
    class Joke(BaseModel):
        setup: str = Field(description="The setup or context of the
joke")
        punchline: str = Field(description="The punchline that delivers
the laugh")
        humor_type: str = Field(description="Type of humor: pun,
observational, dark, etc")

    pydantic_parser = PydanticOutputParser(pydantic_object=Joke)

    prompt = PromptTemplate(
        template="""Generate a joke about {topic}.

        {format_instructions}""",
        input_variables=["topic"],
        partial_variables={
            "format_instructions":
pydantic_parser.get_format_instructions()
        }
    )
```

```
    chain = prompt | llm | pydantic_parser

    result = chain.invoke({"topic": "programming"})
    print(f"Result type: {type(result)}")
    print(f"Setup: {result.setup}")
    print(f"Punchline: {result.punchline}")
    print(f"Humor type: {result.humor_type}")

# ===== EXAMPLE 4: COMMA-SEPARATED LIST PARSER =====

def example_list_parser():
    """Parse comma-separated values into lists"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: COMMA-SEPARATED LIST PARSER")
    print("=" * 70)

    list_parser = CommaSeparatedListOutputParser()

    prompt = PromptTemplate(
        template="""List 5 programming languages suitable for
{use_case},
        separated by commas.

        Format: language1, language2, language3, language4,
language5""",
        input_variables=["use_case"]
    )

    chain = prompt | llm | list_parser

    result = chain.invoke({"use_case": "web development"})
    print(f"Result type: {type(result)}")
    print(f"Result: {result}")

    for i, lang in enumerate(result, 1):
        print(f"  {i}. {lang.strip()}")
```

```
# ===== EXAMPLE 5: COMPLEX PYDANTIC MODELS =====

def example_complex_pydantic():
    """Pydantic parser with nested objects"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: COMPLEX PYDANTIC MODEL")
    print("=" * 70)

    # Nested schema
    class Author(BaseModel):
        name: str
        specialty: str

    class BlogPost(BaseModel):
        title: str = Field(description="Blog post title")
        author: Author = Field(description="Post author")
        tags: list = Field(description="List of tags")
        word_count: int = Field(description="Estimated word count")

    parser = PydanticOutputParser(pydantic_object=BlogPost)

    prompt = PromptTemplate(
        template="""Create a blog post about {topic}.

        {format_instructions}""",
        input_variables=["topic"],
        partial_variables={
            "format_instructions": parser.get_format_instructions()
        }
    )

    chain = prompt | llm | parser

    result = chain.invoke({"topic": "machine learning"})
    print(f"Title: {result.title}")
    print(f"Author: {result.author.name} ({result.author.specialty})")
    print(f"Tags: {', '.join(result.tags)}")
```

```
    print(f"Word count: {result.word_count}")

# ===== EXAMPLE 6: ERROR HANDLING =====

def example_error_handling():
    """Demonstrate error handling in parsing"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: ERROR HANDLING IN PARSING")
    print("=" * 70)

    class Requirement(BaseModel):
        name: str
        priority: int  # Must be int, not string

    parser = PydanticOutputParser(pydantic_object=Requirement)

    # Valid JSON
    print("Test 1: Valid JSON")
    valid_json = json.dumps({"name": "Fix bug", "priority": 1})
    try:
        result = parser.invoke(valid_json)
        print(f"  ✓ Success: {result}")
    except OutputParserException as e:
        print(f"  ✗ Failed: {e}")

    # Invalid JSON (wrong type)
    print("\nTest 2: Invalid type (priority as string)")
    invalid_type = json.dumps({"name": "Fix bug", "priority": "high"})
    try:
        result = parser.invoke(invalid_type)
        print(f"  ✓ Success: {result}")
    except OutputParserException as e:
        print(f"  ✗ Expected error caught")
        print(f"     {str(e)[:100]}...")

    # Malformed JSON
    print("\nTest 3: Malformed JSON")
```

```
    malformed = '{"name": "Fix bug", priority: 1}'  # priority not
quoted
    try:
        result = parser.invoke(malformed)
        print(f"  ✓ Success: {result}")
    except OutputParserException as e:
        print(f"  ✗ Expected error caught")
        print(f"     {str(e)[:100]}...")

# ===== EXAMPLE 7: PARSER COMPOSITION =====

def example_parser_composition():
    """Chain multiple parsers for complex workflows"""
    print("\n" + "=" * 70)
    print("EXAMPLE 7: PARSER COMPOSITION")
    print("=" * 70)

    # Step 1: Get JSON from LLM
    json_parser = JsonOutputParser()

    prompt = PromptTemplate(
        template="""Generate 3 programming tips in JSON format.

        {format_instructions}""",
        input_variables=[],
        partial_variables={
            "format_instructions": json_parser.get_format_instructions()
        }
    )

    chain = prompt | llm | json_parser

    result = chain.invoke({})
    print(f"Parsed output: {result}")
    print(f"Number of tips: {len(result) if isinstance(result, (list,
dict)) else 'N/A'}")
```

```
# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_str_parser()
    example_json_parser()
    example_pydantic_parser()
    example_list_parser()
    example_complex_pydantic()
    example_error_handling()
    example_parser_composition()

    print("\n" + "=" * 70)
    print("✓ All output parsing examples completed")
    print("=" * 70)
```

Detailed Data Workflow Explanation

String Parser Workflow

```
LLM Response (AIMessage):
  {
    "content": "Why did the programmer quit? Because he didn't get
arrays!",
    "role": "assistant",
    ...
  }

StrOutputParser.invoke(response):
  1. Check if response has .content attribute
  2. Extract: response.content
  3. Return as string

Output:
  "Why did the programmer quit? Because he didn't get arrays!"
  (type: str)
```

JSON Parser Workflow

```
LLM Response (AIMessage):
  {
    "content": '{"name": "Alice", "age": 30}',
    ...
  }

JsonOutputParser.invoke(response):
  1. Extract response.content
  2. Validate JSON syntax
  3. Parse with json.loads()
  4. Return dict

Output:
  {"name": "Alice", "age": 30}
  (type: dict)

Access:
  result["name"] → "Alice"
  result["age"] → 30
```

Pydantic Parser Workflow

```
LLM Response (AIMessage):
  {
    "content": '{"setup": "Why...", "punchline": "Because..."}',
    ...
  }

PydanticOutputParser.invoke(response):
  1. Extract response.content
  2. Parse JSON to dict
  3. Create Pydantic model instance
  4. Validate against schema
  5. Return model object

Output:
  Joke(setup="Why...", punchline="Because...")
  (type: Joke model instance)

Access:
  result.setup → "Why..."
  result.punchline → "Because..."

Validation:
  - setup and punchline must be strings
  - Both fields required
  - If LLM omits field → ValidationError
```

Chains & LCEL

Theory & Core Concepts

What is a Chain?

A chain is a sequence of composable operations where output of one becomes input of the next. LCEL (LangChain Expression Language) makes chains easy to build using the `|` (pipe) operator.

Why Chains Matter

1. **Composition**: Build complex applications from simple parts
2. **Reusability**: Chain components can be used in other chains
3. **Readability**: `prompt | llm | parser` is clear and intuitive
4. **Flexibility**: Rearrange components without rewriting code
5. **Streaming Support**: Chains automatically support streaming

Chain Architecture

```
Input
  ↓
[Step 1: Prompt Template]
  ↓
[Step 2: LLM]
  ↓
[Step 3: Output Parser]
  ↓
[Step 4: Custom Function]
  ↓
Output
```

LCEL Pipe Operator

The `|` operator is key to LCEL:

```
# Without LCEL (verbose)
formatted = prompt_template.invoke({"topic": "Python"})
response = llm.invoke(formatted)
parsed = parser.invoke(response)

# With LCEL (elegant)
chain = prompt_template | llm | parser
result = chain.invoke({"topic": "Python"})
```

Runnable Protocol

All LCEL components implement the Runnable protocol:

```
class Runnable:
    def invoke(input):        # Synchronous, single call
        ...

    def stream(input):        # Stream output tokens
        ...

    def batch(inputs):        # Process multiple inputs
        ...

    def __or__(other):        # Support | operator
        return RunnableSequence(self, other)
```

Line-by-Line Code Breakdown

Basic Chain (Prompt → LLM → Parser)

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# Step 1: Create prompt template
prompt_template = PromptTemplate(
    template="Write a joke about {topic}",
    input_variables=["topic"]
)

# Step 2: Create LLM
llm = ChatOpenAI(model="gpt-3.5-turbo")

# Step 3: Create parser
parser = StrOutputParser()

# Step 4: Compose into chain using |
chain = prompt_template | llm | parser
# Why: | operator chains components together
# How it helps: Output of prompt_template → input of llm → output of llm
→ input of parser

# Step 5: Invoke chain
result = chain.invoke({"topic": "Python"})
# Why: invoke() runs entire chain end-to-end
# How it helps: Single call instead of 3 separate invoke() calls
```

Intermediate Transformation

```
def format_response(text: str) -> str:
    """Custom transformation function"""
    return f"The answer is:\n{text}"

# Add custom function to chain
chain = prompt_template | llm | parser | format_response
# Why: | operator works with any callable
# How it helps: Custom logic integrates seamlessly into chain

result = chain.invoke({"topic": "Python"})
# Result includes custom formatting
```

Branching Chains (Parallel Execution)

```
from langchain_core.runnables import RunnableParallel

# Create multiple independent chains
chain1 = prompt1 | llm | parser1
chain2 = prompt2 | llm | parser2

# Run both in parallel
parallel_chain = RunnableParallel(
    result1=chain1,
    result2=chain2
)

results = parallel_chain.invoke({"topic": "Python"})
# Why: RunnableParallel executes chains concurrently
# How it helps: Results as dict with "result1" and "result2" keys
# Output: {"result1": "...", "result2": "..."}
```

Conditional Chains

```
from langchain_core.runnables import RunnableIfElse

# Create condition function
def is_question(input_dict):
    return "?" in input_dict.get("user_input", "")

# Create alternative chains
question_chain = prompt_for_qa | llm | parser
statement_chain = prompt_for_statement | llm | parser

# Conditionally route
conditional = RunnableIfElse(
    condition=is_question,
    if_true=question_chain,
    if_false=statement_chain
)

result = conditional.invoke({"user_input": "What is Python?"})
# Why: Conditional routing based on input
# How it helps: Different processing path for different input types
```

Complete, Ready-to-Run Code Block

```
"""
Complete Chains & LCEL Example
Demonstrates basic chains, parallel execution, and composition
"""

from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser,
JsonOutputParser
from langchain_core.runnables import RunnableParallel,
RunnablePassthrough
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# ===== EXAMPLE 1: BASIC CHAIN =====

def example_basic_chain():
    """Simple prompt → LLM → parser chain"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: BASIC CHAIN (Prompt → LLM → Parser)")
    print("=" * 70)

    # Define components
    prompt = PromptTemplate(
        template="Write a 2-line joke about {topic}",
        input_variables=["topic"]
    )
```

```
    parser = StrOutputParser()

    # Compose chain
    chain = prompt | llm | parser

    # Invoke
    result = chain.invoke({"topic": "Python"})
    print(f"Result:\n{result}")

# ===== EXAMPLE 2: CHAIN WITH CUSTOM FUNCTION =====

def example_chain_with_function():
    """Chain that includes custom transformation function"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: CHAIN WITH CUSTOM FUNCTION")
    print("=" * 70)

    prompt = PromptTemplate(
        template="Generate 3 tips for learning {skill}",
        input_variables=["skill"]
    )

    parser = StrOutputParser()

    # Custom transformation function
    def add_header(text):
        return f"🎯 LEARNING TIPS\n{'=' * 50}\n{text}"

    # Chain with custom function
    chain = prompt | llm | parser | add_header

    result = chain.invoke({"skill": "Python"})
    print(result)

# ===== EXAMPLE 3: MULTI-STEP CHAIN =====

def example_multi_step_chain():
```

```
    """Chain with multiple intermediate steps"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: MULTI-STEP CHAIN")
    print("=" * 70)

    # Step 1: Generate topic
    step1_prompt = PromptTemplate(
        template="Suggest a blog topic related to {interest}",
        input_variables=["interest"]
    )
    step1_parser = StrOutputParser()
    step1_chain = step1_prompt | llm | step1_parser

    # Step 2: Generate outline
    step2_prompt = PromptTemplate(
        template="Create an outline for blog about: {topic}",
        input_variables=["topic"]
    )
    step2_parser = StrOutputParser()
    step2_chain = step2_prompt | llm | step2_parser

    # Execute step 1
    topic = step1_chain.invoke({"interest": "AI"})
    print(f"Generated topic: {topic}\n")

    # Execute step 2 with step 1 output
    outline = step2_chain.invoke({"topic": topic})
    print(f"Generated outline:\n{outline}")

# ===== EXAMPLE 4: PARALLEL CHAINS =====

def example_parallel_chains():
    """Execute multiple chains in parallel"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: PARALLEL CHAINS (RunnableParallel)")
    print("=" * 70)
```

```
    # Chain 1: Generate title
    title_chain = (
        PromptTemplate(
            template="Generate a catchy title about {topic}",
            input_variables=["topic"]
        )
        | llm
        | StrOutputParser()
    )

    # Chain 2: Generate description
    desc_chain = (
        PromptTemplate(
            template="Write a short description about {topic}",
            input_variables=["topic"]
        )
        | llm
        | StrOutputParser()
    )

    # Chain 3: Generate keywords
    keywords_chain = (
        PromptTemplate(
            template="List 5 keywords about {topic} separated by
commas",
            input_variables=["topic"]
        )
        | llm
        | StrOutputParser()
    )

    # Parallel execution
    parallel_chain = RunnableParallel(
        title=title_chain,
        description=desc_chain,
        keywords=keywords_chain
    )
```

```
    result = parallel_chain.invoke({"topic": "Machine Learning"})

    print(f"Title: {result['title']}")
    print(f"\nDescription: {result['description']}")
    print(f"\nKeywords: {result['keywords']}")

# ===== EXAMPLE 5: CHAIN WITH PYDANTIC OUTPUT =====

def example_chain_pydantic():
    """Chain that outputs structured Pydantic object"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: CHAIN WITH PYDANTIC OUTPUT")
    print("=" * 70)

    # Define schema
    class Article(BaseModel):
        title: str = Field(description="Article title")
        author: str = Field(description="Author name")
        summary: str = Field(description="Brief summary")
        keywords: list = Field(description="List of keywords")

    # Create parser
    parser = PydanticOutputParser(pydantic_object=Article)

    # Create prompt
    prompt = PromptTemplate(
        template="""Generate article metadata about {topic}.

        {format_instructions}""",
        input_variables=["topic"],
        partial_variables={
            "format_instructions": parser.get_format_instructions()
        }
    )

    # Chain
```

```
    chain = prompt | llm | parser

    # Invoke
    result = chain.invoke({"topic": "Web Development"})

    # Access structured data
    print(f"Title: {result.title}")
    print(f"Author: {result.author}")
    print(f"Summary: {result.summary}")
    print(f"Keywords: {', '.join(result.keywords)}")

# ===== EXAMPLE 6: CHAIN WITH STREAMING =====

def example_chain_streaming():
    """Chain that supports streaming output"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: CHAIN WITH STREAMING")
    print("=" * 70)

    prompt = PromptTemplate(
        template="Write a poem about {theme}",
        input_variables=["theme"]
    )

    parser = StrOutputParser()
    chain = prompt | llm | parser

    # Stream output
    print("Streaming poem:\n")
    for chunk in chain.stream({"theme": "Python"}):
        print(chunk, end="", flush=True)
    print()

# ===== EXAMPLE 7: CHAIN WITH PASS-THROUGH =====

def example_chain_passthrough():
    """Include original input in chain output"""
```

```
    print("\n" + "=" * 70)
    print("EXAMPLE 7: CHAIN WITH PASS-THROUGH")
    print("=" * 70)

    # Create main chain
    chain = (
        PromptTemplate(
            template="Explain {concept}",
            input_variables=["concept"]
        )
        | llm
        | StrOutputParser()
    )

    # Wrap with pass-through to include original input
    full_chain = RunnableParallel(
        concept=RunnablePassthrough(),
        explanation=chain
    )

    result = full_chain.invoke({"concept": "Decorator Pattern"})

    print(f"Concept: {result['concept']}")
    print(f"Explanation: {result['explanation'][:200]}...")

# ===== EXAMPLE 8: CHAINING WITH LAMBDA =====

def example_chain_lambda():
    """Use lambda for simple transformations"""
    print("\n" + "=" * 70)
    print("EXAMPLE 8: LAMBDA IN CHAINS")
    print("=" * 70)

    from langchain_core.runnables import RunnableLambda

    prompt = PromptTemplate(
        template="Generate a coding tip about {topic}",
```

```
        input_variables=["topic"]
    )

    parser = StrOutputParser()

    # Lambda to add prefix
    add_prefix = RunnableLambda(lambda x: f"💡 {x}")

    # Chain with lambda
    chain = prompt | llm | parser | add_prefix

    result = chain.invoke({"topic": "Python lists"})
    print(result)

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_chain()
    example_chain_with_function()
    example_multi_step_chain()
    example_parallel_chains()
    example_chain_pydantic()
    example_chain_streaming()
    example_chain_passthrough()
    example_chain_lambda()

    print("\n" + "=" * 70)
    print("✓ All chains & LCEL examples completed")
    print("=" * 70)
```

Detailed Data Workflow Explanation

Basic Chain Execution

```
Input: {"topic": "Python"}
  ↓
[Prompt Template]
  template: "Write a joke about {topic}"
  Replaces {topic} with "Python"
  Output: "Write a joke about Python"
  ↓
[LLM]
  Input: "Write a joke about Python"
  API call to model
  Output: AIMessage(content="Why did...Because...")
  ↓
[Output Parser]
  Input: AIMessage(content="Why did...Because...")
  Extract .content
  Output: "Why did...Because..."
  ↓
Final Output: "Why did...Because..."
```

Parallel Chain Execution

```
Input: {"topic": "AI"}
  ├─→ [Chain 1: Title]
  │    └─→ "The Future of AI"
  ├─→ [Chain 2: Description]
  │    └─→ "Artificial intelligence is..."
  └─→ [Chain 3: Keywords]
       └─→ "AI, Machine Learning, Neural..."

Combine Results:
  {
    "title": "The Future of AI",
    "description": "Artificial intelligence is...",
    "keywords": "AI, Machine Learning, Neural..."
  }
```

Stream Output Flow

```
Input: {"theme": "Python"}
  ↓
[Prompt → LLM in streaming mode]

LLM outputs tokens sequentially:
  Chunk 1: "Python"
  Chunk 2: " is"
  Chunk 3: " a"
  Chunk 4: " language"
  ...

[Parser processes each chunk]
  Each chunk → extract → yield

Output stream:
  for chunk in chain.stream(...):
    print(chunk, end="")

Result: "Python is a language..." (displayed in real-time)
```

Structured Output with Pydantic

Theory & Core Concepts

What is Structured Output?

Structured output means converting LLM text responses into strongly-typed Python objects with validation.

Why Pydantic for LLM Outputs?

1. **Validation**: Ensure output matches expected schema
2. **Type Safety**: IDE knows all available fields
3. **Documentation**: Field descriptions guide LLM behavior

4. **Serialization**: Easy conversion to JSON, database rows, etc.
5. **Reusability**: Same schema across multiple prompts

Pydantic vs TypedDict

Feature	Pydantic	TypedDict
Validation	✓ Runtime type checking	✗ Type hints only
Default values	✓ Can provide defaults	✓ Can provide defaults
Error messages	✓ Detailed validation errors	✗ Generic type errors
JSON schema	✓ Auto-generates schema	✗ Manual work
IDE support	✓ Full autocomplete	✓ Full autocomplete

Line-by-Line Code Breakdown

Basic Pydantic Model

```
from pydantic import BaseModel, Field

class Joke(BaseModel):
    setup: str = Field(
        description="The setup or context of the joke"
        # Why description: Guides LLM on what to generate
        # How it helps: Better quality output; LLM knows exact field
purpose
    )

    punchline: str = Field(
        description="The punchline that delivers the joke"
    )

# Create instance
joke = Joke(
    setup="Why did the Python developer go to therapy?",
    punchline="Because of all their issues with indentation!"
)

print(joke.setup)  # Attribute access
print(joke.punchline)
```

Model Validation

```
from pydantic import BaseModel, Field, ValidationError

class Person(BaseModel):
    name: str
    age: int  # Must be integer

# Valid - type matches
try:
    person = Person(name="Alice", age=30)
    # Why: age is int 30
    # How it helps: Passes validation; object created successfully
    print(f"✓ Valid: {person}")
except ValidationError as e:
    print(f"✗ Error: {e}")

# Invalid - wrong type
try:
    person = Person(name="Bob", age="thirty")
    # Why: age is string "thirty", not int
    # How it helps: Validation catches error immediately
    print(f"✓ Valid: {person}")
except ValidationError as e:
    print(f"✗ Error: {e}")
    # Output: "Input should be a valid integer"

# Invalid - missing required field
try:
    person = Person(name="Charlie")  # Missing age
    # Why: age is required (no default)
    # How it helps: Validates all required fields present
    print(f"✓ Valid: {person}")
except ValidationError as e:
    print(f"✗ Error: {e}")
    # Output: "Field required"
```

Nested Models

```
from pydantic import BaseModel, Field

class Address(BaseModel):
    street: str
    city: str
    country: str

class Person(BaseModel):
    name: str
    age: int
    address: Address  # Nested model
    # Why: Compose complex structures from simple models
    # How it helps: Hierarchical validation; reusable Address for other
models

# Create with nested data
person = Person(
    name="Alice",
    age=30,
    address={  # Auto-converts dict to Address
        "street": "123 Main St",
        "city": "New York",
        "country": "USA"
    }
)

# Access nested fields
print(person.address.city)  # "New York"
```

Model with Default Values

```
from pydantic import BaseModel, Field

class Article(BaseModel):
    title: str
    content: str
    author: str = Field(default="Anonymous")
    # Why: Provide default if LLM doesn't generate author
    # How it helps: Makes field optional; prevents validation error if
missing

    tags: list = Field(default_factory=list)
    # Why: Use default_factory for mutable defaults
    # How it helps: Each instance gets fresh list; avoids mutable
default bugs

# Create without optional fields
article = Article(
    title="Python Tips",
    content="Use decorators for..."
)

print(article.author)  # "Anonymous" (default)
print(article.tags)    # [] (empty list from factory)
```

Custom Validation

```
from pydantic import BaseModel, Field, field_validator

class Product(BaseModel):
    name: str
    price: float
    stock: int

    @field_validator('price')
    @classmethod
    def price_must_be_positive(cls, v):
        # Why: Custom validation logic
        # How it helps: Ensures price > 0; rejects nonsensical data
        if v <= 0:
            raise ValueError('Price must be positive')
        return v

    @field_validator('stock')
    @classmethod
    def stock_must_be_non_negative(cls, v):
        if v < 0:
            raise ValueError('Stock cannot be negative')
        return v

# Valid
product1 = Product(name="Laptop", price=999.99, stock=10)
print(f"✓ {product1.name}: ${product1.price}")

# Invalid - negative price
try:
    product2 = Product(name="Phone", price=-500, stock=5)
except Exception as e:
    print(f"✗ Validation failed: {e}")
```

JSON Schema Generation

```
from pydantic import BaseModel, Field
import json

class BookRecommendation(BaseModel):
    title: str = Field(description="Book title")
    author: str = Field(description="Author name")
    rating: float = Field(description="Rating from 1-5")

# Auto-generate JSON schema
schema = BookRecommendation.model_json_schema()
# Why: Generate schema for LLM
# How it helps: Tell LLM exact expected format

print(json.dumps(schema, indent=2))
# Output: JSON schema with all fields, types, descriptions
# Used in PydanticOutputParser.get_format_instructions()
```

Complete, Ready-to-Run Code Block

```
"""
Complete Pydantic Structured Output Example
Demonstrates model definition, validation, and LLM integration
"""

from pydantic import BaseModel, Field, field_validator, ValidationError
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from typing import List, Optional
import json
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# ===== EXAMPLE 1: BASIC PYDANTIC MODEL =====

def example_basic_model():
    """Define and use a simple Pydantic model"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: BASIC PYDANTIC MODEL")
    print("=" * 70)

    class BlogPost(BaseModel):
        title: str = Field(description="Blog post title")
        author: str = Field(description="Author name")
        content: str = Field(description="Post content")
        published: bool = Field(description="Is post published?")

    # Create instance
    post = BlogPost(
```

```
        title="Learning Python",
        author="John Doe",
        content="Python is a great language...",
        published=True
    )

    print(f"Title: {post.title}")
    print(f"Author: {post.author}")
    print(f"Published: {post.published}")

    # Convert to dict
    post_dict = post.model_dump()
    print(f"\nAs dictionary: {post_dict}")

    # Convert to JSON
    post_json = post.model_dump_json(indent=2)
    print(f"\nAs JSON: {post_json}")

# ===== EXAMPLE 2: MODEL WITH VALIDATION =====

def example_model_validation():
    """Demonstrate field validation"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: MODEL WITH VALIDATION")
    print("=" * 70)

    class Product(BaseModel):
        name: str
        price: float
        quantity: int

        @field_validator('price')
        @classmethod
        def price_positive(cls, v):
            if v <= 0:
                raise ValueError('Price must be greater than 0')
            return v
```

```
        @field_validator('quantity')
        @classmethod
        def quantity_non_negative(cls, v):
            if v < 0:
                raise ValueError('Quantity cannot be negative')
            return v

    # Valid data
    print("Test 1: Valid data")
    try:
        product = Product(name="Laptop", price=999.99, quantity=10)
        print(f"  ✓ {product.name}: ${product.price} (qty:
{product.quantity})")
    except ValidationError as e:
        print(f"  ✗ {e}")

    # Invalid price
    print("\nTest 2: Invalid price")
    try:
        product = Product(name="Phone", price=-299, quantity=5)
        print(f"  ✓ {product.name}")
    except ValidationError as e:
        print(f"  ✗ Validation failed: {str(e)[:80]}...")

    # Invalid quantity
    print("\nTest 3: Invalid quantity")
    try:
        product = Product(name="Tablet", price=499, quantity=-2)
        print(f"  ✓ {product.name}")
    except ValidationError as e:
        print(f"  ✗ Validation failed: {str(e)[:80]}...")

# ===== EXAMPLE 3: NESTED MODELS =====

def example_nested_models():
    """Hierarchical model composition"""
```

```
    print("\n" + "=" * 70)
    print("EXAMPLE 3: NESTED MODELS")
    print("=" * 70)

    class Address(BaseModel):
        street: str
        city: str
        state: str
        zip_code: str

    class Person(BaseModel):
        name: str
        email: str
        age: int
        address: Address

    # Create with nested data
    person = Person(
        name="Alice Johnson",
        email="alice@example.com",
        age=30,
        address={
            "street": "123 Main Street",
            "city": "San Francisco",
            "state": "CA",
            "zip_code": "94105"
        }
    )

    print(f"Name: {person.name}")
    print(f"Email: {person.email}")
    print(f"Age: {person.age}")
    print(f"\nAddress:")
    print(f"  {person.address.street}")
    print(f"  {person.address.city}, {person.address.state}
{person.address.zip_code}")
```

```
    # Nested JSON
    print(f"\nJSON:\n{person.model_dump_json(indent=2)}")

# ===== EXAMPLE 4: OPTIONAL FIELDS =====

def example_optional_fields():
    """Models with optional and default values"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: OPTIONAL FIELDS")
    print("=" * 70)

    class UserProfile(BaseModel):
        username: str
        email: str
        full_name: Optional[str] = None  # Optional field
        bio: str = "No bio provided"      # Default value
        tags: List[str] = Field(default_factory=list)

    # Minimal data
    user1 = UserProfile(
        username="alice",
        email="alice@example.com"
    )
    print(f"User 1: {user1.username}")
    print(f"  Full name: {user1.full_name} (None - optional)")
    print(f"  Bio: {user1.bio} (default)")
    print(f"  Tags: {user1.tags} (empty list)")

    # Full data
    user2 = UserProfile(
        username="bob",
        email="bob@example.com",
        full_name="Bob Smith",
        bio="Python developer",
        tags=["python", "coding", "developer"]
    )
    print(f"\nUser 2: {user2.username}")
```

```
    print(f"  Full name: {user2.full_name}")
    print(f"  Bio: {user2.bio}")
    print(f"  Tags: {', '.join(user2.tags)}")

# ===== EXAMPLE 5: JSON SCHEMA =====

def example_json_schema():
    """Generate and inspect JSON schema"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: JSON SCHEMA GENERATION")
    print("=" * 70)

    class MovieReview(BaseModel):
        title: str = Field(description="Movie title")
        rating: float = Field(description="Rating from 1-10", ge=1,
le=10)
        review: str = Field(description="Review text")
        recommend: bool = Field(description="Would you recommend?")

    # Generate schema
    schema = MovieReview.model_json_schema()

    print("Generated JSON Schema:")
    print(json.dumps(schema, indent=2))

    print("\n✓ This schema is used by PydanticOutputParser")
    print("  to instruct LLM on expected format")

# ===== EXAMPLE 6: LLM INTEGRATION =====

def example_llm_integration():
    """Use Pydantic model with LLM for structured output"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: LLM INTEGRATION (Structured Output)")
    print("=" * 70)

    class NewsArticle(BaseModel):
```

```
        headline: str = Field(description="Article headline")
        summary: str = Field(description="Brief summary")
        category: str = Field(description="News category")
        importance: int = Field(description="Importance level 1-10")

    # Create parser
    parser = PydanticOutputParser(pydantic_object=NewsArticle)

    # Create prompt with format instructions
    prompt = PromptTemplate(
        template="""Create a news article about {topic}.

        {format_instructions}""",
        input_variables=["topic"],
        partial_variables={
            "format_instructions": parser.get_format_instructions()
        }
    )

    # Create chain
    chain = prompt | llm | parser

    # Execute
    article = chain.invoke({"topic": "AI advancement"})

    print(f"Headline: {article.headline}")
    print(f"Summary: {article.summary}")
    print(f"Category: {article.category}")
    print(f"Importance: {article.importance}/10")

# ===== EXAMPLE 7: COMPLEX MODELS =====

def example_complex_model():
    """Advanced model with multiple nested types"""
    print("\n" + "=" * 70)
    print("EXAMPLE 7: COMPLEX MODEL")
    print("=" * 70)
```

```
    class Author(BaseModel):
        name: str
        email: str

    class Comment(BaseModel):
        author: str
        text: str
        rating: int = Field(ge=1, le=5)

    class BlogPost(BaseModel):
        title: str
        author: Author
        content: str
        tags: List[str]
        comments: List[Comment] = Field(default_factory=list)

    # Create complex object
    post = BlogPost(
        title="Advanced Pydantic",
        author={"name": "John Doe", "email": "john@example.com"},
        content="This post covers advanced Pydantic patterns...",
        tags=["pydantic", "validation", "python"],
        comments=[
            {"author": "Alice", "text": "Great post!", "rating": 5},
            {"author": "Bob", "text": "Very helpful", "rating": 4}
        ]
    )

    print(f"Title: {post.title}")
    print(f"Author: {post.author.name} ({post.author.email})")
    print(f"Tags: {', '.join(post.tags)}")
    print(f"\nComments:")
    for comment in post.comments:
        print(f"  {comment.author}: {comment.text} (⭐
{comment.rating})")
```

```
# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_model()
    example_model_validation()
    example_nested_models()
    example_optional_fields()
    example_json_schema()
    example_llm_integration()
    example_complex_model()

    print("\n" + "=" * 70)
    print("✓ All Pydantic examples completed")
    print("=" * 70)
```

Tools & Tool Binding

Theory & Core Concepts

What are Tools?

Tools are functions/APIs that an LLM can call to interact with the outside world (search engines, calculators, databases, APIs, etc.).

Why Tools Matter

1. **Extended Capabilities**: LLM can perform actions, not just generate text
2. **Real Data**: Access current information (weather, stocks, news)
3. **External Systems**: Integrate with databases, APIs, microservices
4. **Grounding**: Reduce hallucinations by using real data
5. **Agentic Behavior**: Foundation for autonomous AI agents

Tool Architecture

```
LLM
  ├─ Bound with tools
  └─ Can call: Tool1, Tool2, Tool3

When LLM wants to use tool:
  LLM → "I need to use search_tool with query='Python'"
  Agent → Call search_tool('Python')
  Agent → Get result
  Agent → Pass result back to LLM
  LLM → Continue reasoning with result
```

Tool Types

1. **Built-in Tools**: Provided by LangChain (Wikipedia search, DuckDuckGo)
2. **Custom Tools**: Write your own function + decorator
3. **Third-party APIs**: Wrapped in LangChain tool format
4. **Python Functions**: Any callable can be a tool

Line-by-Line Code Breakdown

Basic Tool Creation

```
from langchain_core.tools import tool

@tool
def calculator(expression: str) -> str:
    """Calculate a mathematical expression

    Args:
        expression: Math expression to evaluate (e.g., '2+2')
    """
    try:
        result = eval(expression)
        # Why: eval() evaluates math expressions
        # How it helps: Provides answer to LLM
        return str(result)
    except Exception as e:
        return f"Error: {e}"

# Why @tool decorator: Converts function to LangChain Tool object
# How it helps: Automatically handles schema generation for LLM

# Tool is now callable
answer = calculator("5 * 3")
print(answer)  # "15"

# Tool has schema for LLM
print(calculator.name)       # "calculator"
print(calculator.description) # Docstring
```

Tool Binding to LLM

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

# Create tools
@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

llm = ChatOpenAI(model="gpt-3.5-turbo")

# Bind tools to LLM
llm_with_tools = llm.bind_tools([add, multiply])
# Why: bind_tools() tells LLM about available tools
# How it helps: LLM can choose to call tools when helpful

# Now LLM can return tool calls
response = llm_with_tools.invoke("What is 5 times 3?")
# LLM recognizes it can use multiply tool
# response contains tool call information
```

Handling Tool Calls

```
from langchain_core.messages import HumanMessage
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get weather for a city"""
    return f"Weather in {city}: Sunny, 72°F"

llm_with_tools = llm.bind_tools([get_weather])

# Invoke
response = llm_with_tools.invoke([
    HumanMessage(content="What's the weather in Paris?")
])

# Response contains tool_calls attribute
if response.tool_calls:
    for tool_call in response.tool_calls:
        tool_name = tool_call["name"]  # "get_weather"
        tool_args = tool_call["args"]  # {"city": "Paris"}
        # Why: Extract tool info from response
        # How it helps: Execute the actual tool function

# Execute tool
if tool_name == "get_weather":
    result = get_weather(tool_args["city"])
    print(result)  # "Weather in Paris: Sunny, 72°F"
```

Built-in Tools from LangChain

```
from langchain_community.tools import DuckDuckGoSearchRun,
WikipediaQueryRun
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper,
WikipediaAPIWrapper

# Create Wikipedia search tool
wikipedia = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
# Why: Wraps Wikipedia API as LangChain Tool
# How it helps: LLM can search Wikipedia

# Create DuckDuckGo search tool
search = DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper())
# Why: Wraps web search
# How it helps: LLM accesses current web information

# Use tools
result = wikipedia.invoke("Python programming language")
# Returns: Wikipedia article summary

# Bind to LLM
llm_with_search = llm.bind_tools([wikipedia, search])
```

Complete, Ready-to-Run Code Block

```
"""
Complete Tools & Tool Binding Example
Demonstrates tool creation, binding, and execution
"""

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_community.utilities import DuckDuckGoSearchAPIWrapper
from typing import Union
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# ===== EXAMPLE 1: BASIC TOOL CREATION =====

def example_basic_tool():
    """Create and use a simple tool"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: BASIC TOOL CREATION")
    print("=" * 70)

    @tool
    def calculator(expression: str) -> str:
        """Evaluate a mathematical expression

        Args:
            expression: Math expression to evaluate (e.g. '5+3', '10*2')

        Returns:
```

```
            String representation of the result
        """
        try:
            result = eval(expression)
            return str(result)
        except Exception as e:
            return f"Error: {str(e)[:50]}"

    # Use tool directly
    print("Direct call: calculator('5 * 3')")
    print(f"  Result: {calculator('5 * 3')}")

    # Inspect tool schema
    print(f"\nTool name: {calculator.name}")
    print(f"Tool description: {calculator.description[:80]}...")
    print(f"Tool args schema: {calculator.args}")

# ===== EXAMPLE 2: MULTIPLE TOOLS =====

def example_multiple_tools():
    """Create and manage multiple tools"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: MULTIPLE TOOLS")
    print("=" * 70)

    @tool
    def add(a: float, b: float) -> float:
        """Add two numbers"""
        return a + b

    @tool
    def subtract(a: float, b: float) -> float:
        """Subtract second from first"""
        return a - b

    @tool
    def multiply(a: float, b: float) -> float:
```

```
        """Multiply two numbers"""
        return a * b

    @tool
    def divide(a: float, b: float) -> Union[float, str]:
        """Divide first by second"""
        if b == 0:
            return "Error: Division by zero"
        return a / b

    # Tool list
    math_tools = [add, subtract, multiply, divide]

    print("Available math tools:")
    for tool in math_tools:
        print(f"  - {tool.name}: {tool.description}")

    # Direct usage
    print(f"\nDirect calls:")
    print(f"  add(10, 5) = {add(10, 5)}")
    print(f"  multiply(4, 3) = {multiply(4, 3)}")
    print(f"  divide(20, 4) = {divide(20, 4)}")

# ===== EXAMPLE 3: TOOL BINDING =====

def example_tool_binding():
    """Bind tools to LLM"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: TOOL BINDING")
    print("=" * 70)

    @tool
    def get_stock_price(symbol: str) -> str:
        """Get current stock price for a symbol"""
        # Simulated data
        prices = {
            "AAPL": "$150.25",
```

```
            "GOOGL": "$140.50",
            "MSFT": "$370.20"
        }
        return prices.get(symbol, "Symbol not found")

    @tool
    def calculate_tax(amount: float, tax_rate: float) -> float:
        """Calculate tax on an amount"""
        return amount * tax_rate

    # Bind tools to LLM
    tools = [get_stock_price, calculate_tax]
    llm_with_tools = llm.bind_tools(tools)

    print("Tools bound to LLM")
    print(f"Available tools: {[t.name for t in tools]}")

    # Query LLM
    response = llm_with_tools.invoke([
        HumanMessage(content="What is the AAPL stock price?")
    ])

    # Check if LLM called a tool
    if response.tool_calls:
        print(f"\nLLM triggered tool calls:")
        for call in response.tool_calls:
            print(f"  - {call['name']}: {call['args']}")
    else:
        print(f"\nLLM response: {response.content[:100]}...")

# ===== EXAMPLE 4: TOOL EXECUTION LOOP =====

def example_tool_execution_loop():
    """Demonstrate agent loop with tool execution"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: TOOL EXECUTION LOOP")
    print("=" * 70)
```

```
@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

# Bind tools
tools = [add, multiply]
tool_map = {tool.name: tool for tool in tools}
llm_with_tools = llm.bind_tools(tools)

# Initial query
messages = [
    HumanMessage(content="Calculate: (5 + 3) * 2")
]

print("Query: (5 + 3) * 2\n")

# Agentic loop
iteration = 0
max_iterations = 5

while iteration < max_iterations:
    iteration += 1

    # Get LLM response
    response = llm_with_tools.invoke(messages)

    # Check for tool calls
    if not response.tool_calls:
        # No tools called - LLM has final answer
        print(f"\nFinal answer: {response.content}")
```

```
            break

        # Execute tools
        print(f"Iteration {iteration}:")
        for tool_call in response.tool_calls:
            tool_name = tool_call['name']
            tool_args = tool_call['args']

            print(f"  Tool: {tool_name}")
            print(f"  Args: {tool_args}")

            # Execute tool
            tool_result = tool_map[tool_name](**tool_args)
            print(f"  Result: {tool_result}\n")

            # Add tool message to conversation
            messages.append(response)
            messages.append(ToolMessage(
                tool_call_id=tool_call['id'],
                content=str(tool_result)
            ))

# ===== EXAMPLE 5: BUILT-IN TOOLS =====

def example_builtin_tools():
    """Use built-in LangChain tools"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: BUILT-IN TOOLS (Web Search)")
    print("=" * 70)

    # Create search tool
    search =
DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper())

    print("Tool name:", search.name)
    print("Tool description:", search.description[:80], "...")
```

```
    # Use tool directly
    try:
        result = search.run("Python programming language")
        print(f"\nSearch result:\n{result[:200]}...")
    except Exception as e:
        print(f"Note: Search requires internet. Error: {str(e)[:50]}")

# ===== EXAMPLE 6: CUSTOM TOOL WITH VALIDATION =====

def example_tool_with_validation():
    """Tool with input validation"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: TOOL WITH VALIDATION")
    print("=" * 70)

    @tool
    def calculate_discount(price: float, discount_percent: float) ->
str:
        """Calculate discounted price

        Args:
            price: Original price (must be positive)
            discount_percent: Discount percentage (0-100)
        """
        if price < 0:
            return "Error: Price cannot be negative"
        if not (0 <= discount_percent <= 100):
            return "Error: Discount must be between 0 and 100"

        discount_amount = price * (discount_percent / 100)
        final_price = price - discount_amount
        return f"Original: ${price:.2f} → Final: ${final_price:.2f}
(Save ${discount_amount:.2f})"

    # Test cases
    print("Test 1: Valid input")
    print(f"  {calculate_discount(100, 20)}")
```

```
    print("\nTest 2: Invalid price")
    print(f"  {calculate_discount(-50, 10)}")

    print("\nTest 3: Invalid discount")
    print(f"  {calculate_discount(100, 150)}")

# ===== EXAMPLE 7: TOOL DOCUMENTATION =====

def example_tool_documentation():
    """Demonstrate tool documentation"""
    print("\n" + "=" * 70)
    print("EXAMPLE 7: TOOL DOCUMENTATION")
    print("=" * 70)

    @tool
    def analyze_text(text: str, analysis_type: str = "summary") -> str:
        """Analyze text for various aspects

        Args:
            text: The text to analyze
            analysis_type: Type of analysis - 'summary', 'sentiment',
'entities'

        Returns:
            Analysis result as a string

        Example:
            >>> analyze_text("Python is great!", "sentiment")
            "Positive sentiment detected"
        """
        if analysis_type == "summary":
            return f"Text length: {len(text)} characters"
        elif analysis_type == "sentiment":
            return "Positive" if len(text) > 10 else "Neutral"
        else:
            return "Unknown analysis type"
```

```
    # Display documentation
    print(f"Tool: {analyze_text.name}")
    print(f"Description:\n{analyze_text.description}")
    print(f"Args schema:\n{analyze_text.args}")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_tool()
    example_multiple_tools()
    example_tool_binding()
    example_tool_execution_loop()
    example_builtin_tools()
    example_tool_with_validation()
    example_tool_documentation()

    print("\n" + "=" * 70)
    print("✓ All tools & tool binding examples completed")
    print("=" * 70)
```

Detailed Data Workflow Explanation

Tool Binding Workflow

```
Step 1: Create Tools
  @tool
  def calculator(expr) → "15"

Step 2: Bind to LLM
  llm_with_tools = llm.bind_tools([calculator])

Step 3: LLM Receives Query
  User: "What is 5 * 3?"

Step 4: LLM Decides to Use Tool
  Internal reasoning: "I should use calculator tool"

Step 5: Tool Call in Response
  response.tool_calls = [{
    "name": "calculator",
    "args": {"expr": "5*3"},
    "id": "call_123"
  }]

Step 6: Application Executes Tool
  result = calculator("5*3")
  result = 15

Step 7: Return Result to LLM
  ToolMessage(content="15", tool_call_id="call_123")

Step 8: LLM Continues
  Final output: "The answer is 15"
```

Tool Execution Loop

```
Initial: messages = [HumanMessage("Calculate (5+3)*2")]

Loop Iteration 1:
  response = llm.invoke(messages)
  → LLM calls: add(5, 3)
  → Add result to messages
  messages.append(ToolMessage(content="8"))

Loop Iteration 2:
  response = llm.invoke(messages)
  → LLM calls: multiply(8, 2)
  → Add result to messages
  messages.append(ToolMessage(content="16"))

Loop Iteration 3:
  response = llm.invoke(messages)
  → No tool calls
  → LLM returns: "The answer is 16"
  → Break loop

Output: 16
```

Agents & Agentic AI

Theory & Core Concepts

What is an Agent?

An agent is an autonomous system that:

1. Takes input/query from user
2. Reasons about available tools
3. Decides which tool(s) to use
4. Executes tools
5. Processes results

6. Repeats until goal achieved or max steps reached

Why Agents Matter

1. **Autonomy**: LLM decides what to do, not following pre-programmed logic
2. **Adaptability**: Handles unexpected scenarios by reasoning
3. **Complex Tasks**: Breaks down multi-step problems automatically
4. **Tool Orchestration**: Coordinates multiple tool calls
5. **Loop Control**: Stops when task is complete or problem is unsolvable

Agent Architecture

```
User Query
    ↓
Agent Reasoning Loop:
  ├─ Observe: Current state, available tools
  ├─ Think: What should I do? Which tool?
  ├─ Act: Call the tool
  ├─ Observe: Tool result
  └─ Loop back to Think or Return Answer
    ↓
Final Answer
```

Agent Patterns

1. **ReAct**: Reasoning + Acting (LLM decides each step)
2. **Plan-Execute**: Plan first, then execute steps
3. **Tool-Use**: Primarily for tool selection
4. **Conversational**: Maintain context across turns

Line-by-Line Code Breakdown

Basic Agent Creation

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

# Create tools
@tool
def calculator(expr: str) -> str:
    """Evaluate math expressions"""
    return str(eval(expr))

tools = [calculator]

# Create LLM
llm = ChatOpenAI(model="gpt-3.5-turbo")

# Create ReAct agent
agent = create_react_agent(llm, tools)
# Why: ReAct = Reasoning + Acting pattern
# How it helps: LLM reasons through steps explicitly

# Create executor (runs the agent loop)
executor = AgentExecutor.from_agent_and_tools(
    agent=agent,
    tools=tools,
    max_iterations=10,
    # Why: Prevent infinite loops
    # How it helps: Agent stops after 10 steps
)

# Run agent
result = executor.invoke({"input": "Calculate 5 * 3 + 2"})
# Why: invoke() runs agent loop until complete
# How it helps: Handles all tool calls and reasoning internally
```

Agent with Streaming

```
from langchain.agents import AgentExecutor

executor = AgentExecutor(...)

# Stream agent output
for step in executor.stream({"input": "What is 2+2?"}):
    if "actions" in step:
        # Tool execution step
        print(f"Tool called: {step['actions']}")
    elif "observations" in step:
        # Tool result
        print(f"Result: {step['observations']}")
    elif "final_output" in step:
        # Final answer
        print(f"Answer: {step['final_output']}")
# Why: Streaming shows progress in real-time
# How it helps: Better UX; can show thinking process
```

Agent Reasoning Process

```
# Enable verbose output to see reasoning
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    # Why: verbose=True prints each thinking step
    # How it helps: Debug and understand agent behavior
)

result = executor.invoke({"input": "query"})

# Output shows:
# Thought: I need to use calculator tool
# Action: calculator
# Action Input: "5 * 3"
# Observation: 15
# Thought: I have the answer
# Final Answer: 15
```

Complete, Ready-to-Run Code Block

```
"""
Complete Agents & Agentic AI Example
Demonstrates agent creation, execution, and streaming
"""

from langchain.agents import create_react_agent, AgentExecutor,
create_tool_calling_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate
from typing import Union
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

# ===== EXAMPLE 1: BASIC REACT AGENT =====

def example_basic_react_agent():
    """Simple ReAct agent with math tools"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: BASIC REACT AGENT")
    print("=" * 70)

    # Define tools
    @tool
    def add(a: int, b: int) -> int:
        """Add two numbers"""
        return a + b

    @tool
    def multiply(a: int, b: int) -> int:
        """Multiply two numbers"""
        return a * b
```

```
    # Create LLM and agent
    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [add, multiply]

    # Create ReAct agent
    agent = create_react_agent(llm, tools)
    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
        max_iterations=10,
        verbose=False
    )

    # Run agent
    query = "What is (5 + 3) * 2?"
    print(f"Query: {query}\n")

    result = executor.invoke({"input": query})
    print(f"Final Answer: {result['output']}")

# ===== EXAMPLE 2: AGENT WITH WEB SEARCH =====

def example_agent_with_search():
    """Agent with web search capability"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: AGENT WITH SEARCH")
    print("=" * 70)

    from langchain_community.tools import DuckDuckGoSearchRun
    from langchain_community.utilities import DuckDuckGoSearchAPIWrapper

    @tool
    def calculator(expression: str) -> str:
        """Evaluate mathematical expressions"""
        try:
            return str(eval(expression))
```

```
        except:
            return "Error evaluating expression"

    # Create search tool
    search =
DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper())

    # Create agent
    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [calculator, search]

    agent = create_tool_calling_agent(llm, tools,
ChatPromptTemplate.from_messages([
        ("system", "You are a helpful assistant"),
        ("human", "{input}"),
    ]))

    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
        max_iterations=5,
        verbose=False
    )

    print("Agent ready with: calculator, search\n")
    print("Query: 'What is the capital of France and its population?'")

    try:
        result = executor.invoke({
            "input": "What is the capital of France and its population?"
        })
        print(f"\nAnswer: {result['output'][:200]}...")
    except Exception as e:
        print(f"(Search requires internet. Error: {str(e)[:50]})")

# ===== EXAMPLE 3: AGENT WITH STREAMING =====
```

```
def example_agent_streaming():
    """Stream agent execution to show thinking process"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: AGENT WITH STREAMING")
    print("=" * 70)

    @tool
    def get_weather(city: str) -> str:
        """Get weather for a city"""
        weather_data = {
            "New York": "Sunny, 72°F",
            "London": "Rainy, 55°F",
            "Paris": "Cloudy, 60°F"
        }
        return weather_data.get(city, "Weather data not available")

    @tool
    def get_time(timezone: str) -> str:
        """Get current time in timezone"""
        times = {
            "EST": "3:30 PM",
            "GMT": "8:30 PM",
            "CET": "9:30 PM"
        }
        return times.get(timezone, "Timezone not found")

    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [get_weather, get_time]

    agent = create_tool_calling_agent(
        llm,
        tools,
        ChatPromptTemplate.from_messages([
            ("system", "You are a helpful assistant"),
            ("human", "{input}"),
        ])
    )
```

```
    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
        max_iterations=10
    )

    print("Streaming agent execution:\n")

    query = "What's the weather in Paris and time in CET?"
    step_count = 0

    for step in executor.stream({"input": query}):
        step_count += 1

        if "actions" in step:
            actions = step["actions"]
            for action in actions:
                print(f"Step {step_count}: Calling {action.tool}")
        elif "observations" in step:
            observations = step["observations"]
            print(f"  Result: {observations}\n")
        elif "final_output" in step:
            print(f"\n✓ Final: {step['final_output']}")

# ===== EXAMPLE 4: AGENT WITH CUSTOM REASONING =====

def example_agent_custom_prompt():
    """Agent with custom system prompt"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: AGENT WITH CUSTOM REASONING")
    print("=" * 70)

    @tool
    def summarize(text: str) -> str:
        """Summarize text"""
        words = text.split()
```

```
        if len(words) <= 10:
            return text
        return f"{' '.join(words[:10])}... (truncated)"

    @tool
    def translate(text: str, language: str) -> str:
        """Translate text (simulated)"""
        return f"[{language.upper()}] {text}"

    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [summarize, translate]

    # Custom prompt with system message
    custom_prompt = ChatPromptTemplate.from_messages([
        ("system", """You are an expert language processor.
Your goal is to understand text and apply transformations carefully.
Always think through each step before taking action.
If uncertain, ask for clarification."""),
        ("human", "{input}")
    ])

    agent = create_tool_calling_agent(llm, tools, custom_prompt)
    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
        max_iterations=5
    )

    query = "Summarize and translate to Spanish: Python is a programming
language"
    print(f"Query: {query}\n")

    result = executor.invoke({"input": query})
    print(f"Result: {result['output']}")

# ===== EXAMPLE 5: AGENT WITH ERROR HANDLING =====
```

```
def example_agent_error_handling():
    """Agent that handles tool errors gracefully"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: AGENT WITH ERROR HANDLING")
    print("=" * 70)

    @tool
    def divide(a: float, b: float) -> Union[float, str]:
        """Divide a by b, handle division by zero"""
        if b == 0:
            return "ERROR: Cannot divide by zero"
        return a / b

    @tool
    def validate_number(value: str) -> Union[float, str]:
        """Validate and convert to number"""
        try:
            return float(value)
        except:
            return f"ERROR: '{value}' is not a valid number"

    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [divide, validate_number]

    agent = create_tool_calling_agent(
        llm,
        tools,
        ChatPromptTemplate.from_messages([
            ("system", "You handle calculations. If an error occurs, try
alternative approaches."),
            ("human", "{input}")
        ])
    )

    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
```

```
        max_iterations=5
    )

    # Test error handling
    test_cases = [
        "Divide 10 by 2",
        "Divide 10 by 0",
        "Validate the number abc"
    ]

    for query in test_cases:
        print(f"\nQuery: {query}")
        result = executor.invoke({"input": query})
        print(f"Result: {result['output'][:100]}")

# ===== EXAMPLE 6: MULTI-STEP AGENT TASK =====

def example_multi_step_task():
    """Agent performs multi-step reasoning"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: MULTI-STEP TASK")
    print("=" * 70)

    @tool
    def get_product_price(product: str) -> str:
        """Get price of product"""
        prices = {"Laptop": 1000, "Mouse": 25, "Keyboard": 75}
        price = prices.get(product, None)
        return str(price) if price else "Product not found"

    @tool
    def apply_discount(price: float, discount_percent: int) -> float:
        """Apply discount to price"""
        return price * (1 - discount_percent / 100)

    @tool
    def calculate_shipping(price: float, weight_kg: float) -> float:
```

```
        """Calculate shipping cost"""
        base_shipping = 10
        weight_charge = weight_kg * 5
        return base_shipping + weight_charge

    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    tools = [get_product_price, apply_discount, calculate_shipping]

    agent = create_tool_calling_agent(
        llm,
        tools,
        ChatPromptTemplate.from_messages([
            ("system", "You are a pricing assistant. Calculate final
cost including discounts and shipping."),
            ("human", "{input}")
        ])
    )

    executor = AgentExecutor.from_agent_and_tools(
        agent=agent,
        tools=tools,
        max_iterations=10
    )

    query = "What's the final cost of a laptop with 10% discount and 2kg
shipping?"
    print(f"Query: {query}\n")

    result = executor.invoke({"input": query})
    print(f"Final Answer:\n{result['output']}")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_react_agent()
    example_agent_with_search()
    example_agent_streaming()
```

```
    example_agent_custom_prompt()
    example_agent_error_handling()
    example_multi_step_task()

    print("\n" + "=" * 70)
    print("✓ All agent examples completed")
    print("=" * 70)
```

RAG & ReAct Pattern

Theory & Core Concepts

What is RAG?

RAG (Retrieval-Augmented Generation) combines:

1. **Retrieval**: Fetch relevant documents from a knowledge base
2. **Augmentation**: Add retrieved context to the prompt
3. **Generation**: LLM generates response using augmented prompt

Why RAG Matters

1. **Reduces Hallucinations**: Ground responses in real data
2. **Current Information**: Access knowledge base, not just training data
3. **Cost Efficient**: Smaller models with RAG outperform larger models
4. **Domain Specific**: Use private documents, APIs, databases
5. **Verifiable**: User can check source documents

ReAct Pattern

ReAct (Reasoning + Acting) is how agents solve problems:

1. **Reason**: "I need to search for information about X"
2. **Act**: Perform the search
3. **Reason**: "Based on the search results, I can now answer..."
4. **Loop**: Repeat until goal achieved

RAG Architecture

```
User Query
    ↓
[Retrieve]
  Query embeddings
  Search vector store
  Get top K relevant documents
    ↓
[Augment]
  Insert documents into prompt
  "Context: {documents}
   Question: {query}"
    ↓
[Generate]
  LLM processes augmented prompt
  Generates response grounded in documents
    ↓
Final Answer with Sources
```

Line-by-Line Code Breakdown

Basic RAG Setup

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

# Step 1: Load documents
documents = [
    Document(page_content="Python is a programming language"),
    Document(page_content="Java is used for enterprise applications"),
]
# Why: Create Document objects for vector storage
# How it helps: Structured format for embeddings and retrieval

# Step 2: Split long documents into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,  # Size of each chunk
    chunk_overlap=20,  # Overlap between chunks
)
# Why: LLMs have token limits; chunks fit within limits
# How it helps: Retrieve specific relevant chunks, not entire documents

chunks = splitter.split_documents(documents)

# Step 3: Create embeddings
embeddings = OpenAIEmbeddings()
# Why: Convert text to numerical vectors for similarity search
# How it helps: Find similar chunks using vector distance

# Step 4: Store in vector database
vector_store = Chroma.from_documents(chunks, embeddings)
# Why: Fast similarity search over many documents
# How it helps: Retrieval in milliseconds even with 1M+ documents

# Step 5: Create retriever
retriever = vector_store.as_retriever(
    search_kwargs={"k": 3}  # Return top 3 similar chunks
)
```

```
# Why: Interface for retrieving relevant documents
# How it helps: Used in RAG chain to get context
```

RAG Chain

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

# Create RAG prompt
rag_prompt = PromptTemplate(
    template="""Answer based on this context:

    Context: {context}

    Question: {question}""",
    input_variables=["context", "question"]
)

# Create LLM
llm = ChatOpenAI(model="gpt-3.5-turbo")

# Build RAG chain
rag_chain = (
    {"context": retriever, "question": IdentityRunnable()}
    | rag_prompt
    | llm
)
# Why: retriever outputs context for "context" variable
# How it helps: Automatic retrieval + prompt formatting + LLM call

# Use chain
result = rag_chain.invoke({"question": "What is Python?"})
```

Complete, Ready-to-Run Code Block

```
"""
Complete RAG Example
Demonstrates vector storage, retrieval, and augmented generation
"""

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_community.vectorstores import FAISS
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

# ===== EXAMPLE 1: BASIC RAG SETUP =====

def example_basic_rag():
    """Basic RAG pipeline"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: BASIC RAG SETUP")
    print("=" * 70)

    # Sample documents about Python
    documents = [
        Document(page_content="""Python is a high-level programming
language.
        It emphasizes code readability and simplicity. Python is widely
used in
        web development, data science, artificial intelligence, and
automation."""),

        Document(page_content="""Python was created by Guido van Rossum
```

```
in 1991.
        The language name is inspired by the British comedy series Monty
Python.
        Python 3 was released in 2008 with major improvements."""),

        Document(page_content="""Popular Python libraries include NumPy
for numerical
        computing, Pandas for data analysis, Django for web frameworks,
and TensorFlow
        for machine learning."""),
    ]

    # Split documents into chunks
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=150,
        chunk_overlap=30
    )
    chunks = splitter.split_documents(documents)
    print(f"Created {len(chunks)} chunks from {len(documents)}
documents")

    # Create embeddings
    embeddings = OpenAIEmbeddings()

    # Create vector store (using FAISS - fast)
    vector_store = FAISS.from_documents(chunks, embeddings)
    print(f"Vector store created with {len(chunks)} chunks")

    # Create retriever
    retriever = vector_store.as_retriever(
        search_kwargs={"k": 2}
    )

    # Test retrieval
    query = "What is Python used for?"
    retrieved = retriever.invoke(query)
```

```
    print(f"\nQuery: {query}")
    print(f"Retrieved {len(retrieved)} relevant chunks:")
    for i, doc in enumerate(retrieved, 1):
        print(f"\n  Chunk {i}:")
        print(f"    {doc.page_content[:100]}...")

# ===== EXAMPLE 2: RAG WITH LLM =====

def example_rag_with_llm():
    """RAG chain with LLM generation"""
    print("\n" + "=" * 70)
    print("EXAMPLE 2: RAG WITH LLM GENERATION")
    print("=" * 70)

    # Documents
    documents = [
        Document(page_content="""Machine Learning is a subset of AI.
        It enables systems to learn from data without explicit
programming.
        ML algorithms improve performance through experience."""),

        Document(page_content="""Common ML algorithms include:
        - Supervised: Linear Regression, Decision Trees, Neural Networks
        - Unsupervised: K-Means, PCA, Hierarchical Clustering
        - Reinforcement: Q-Learning, Policy Gradient"""),
    ]

    # Setup
    splitter = RecursiveCharacterTextSplitter(chunk_size=150,
chunk_overlap=30)
    chunks = splitter.split_documents(documents)

    embeddings = OpenAIEmbeddings()
    vector_store = FAISS.from_documents(chunks, embeddings)
    retriever = vector_store.as_retriever(search_kwargs={"k": 2})

    # RAG Prompt
```

```
    rag_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an AI expert. Answer based on the provided
context."),
        ("human", """Context from knowledge base:
        {context}

        Question: {question}

        Provide a clear answer based on the context.""")
    ])

    # Create LLM and chain
    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
    parser = StrOutputParser()

    # Format context for RAG
    def format_docs(docs):
        return "\n\n".join([doc.page_content for doc in docs])

    # Build chain
    rag_chain = (
        {"context": retriever | format_docs, "question": lambda x:
x["question"]}
        | rag_prompt
        | llm
        | parser
    )

    # Execute
    query = "What are the main types of machine learning?"
    print(f"Query: {query}\n")

    result = rag_chain.invoke({"question": query})
    print(f"Answer:\n{result}")

# ===== EXAMPLE 3: MULTI-DOCUMENT RAG =====
```

```
def example_multi_document_rag():
    """RAG with multiple documents"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: MULTI-DOCUMENT RAG")
    print("=" * 70)

    # Multiple documents
    docs = {
        "Python": Document(page_content="""Python Features:
        - Dynamic typing and automatic memory management
        - Extensive standard library
        - Cross-platform compatibility
        - Strong community support"""),

        "JavaScript": Document(page_content="""JavaScript Features:
        - Runs in browsers and servers (Node.js)
        - Event-driven and asynchronous
        - DOM manipulation for web interfaces
        - JSON is native format"""),

        "Java": Document(page_content="""Java Features:
        - Compiled and platform independent (JVM)
        - Strong type system
        - Enterprise frameworks (Spring)
        - Multi-threading support"""),
    }

    # Prepare documents
    documents = list(docs.values())
    splitter = RecursiveCharacterTextSplitter(chunk_size=100,
chunk_overlap=20)
    chunks = splitter.split_documents(documents)

    # Create retriever
    embeddings = OpenAIEmbeddings()
    vector_store = FAISS.from_documents(chunks, embeddings)
    retriever = vector_store.as_retriever(search_kwargs={"k": 3})
```

```
    # Create RAG chain
    rag_prompt = PromptTemplate(
        template="""Based on this context, answer the question:

        Context:
        {context}

        Question: {question}""",
        input_variables=["context", "question"]
    )

    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

    def format_context(docs):
        return "\n\n".join([d.page_content for d in docs])

    chain = (
        {"context": retriever | format_context, "question": lambda x: x}
        | rag_prompt
        | llm
        | StrOutputParser()
    )

    # Test queries
    queries = [
        "Which language runs in browsers?",
        "What language is used for enterprise applications?",
        "Which language has automatic memory management?"
    ]

    for query in queries:
        print(f"\nQ: {query}")
        result = chain.invoke(query)
        print(f"A: {result[:100]}...")

# ===== EXAMPLE 4: SOURCE ATTRIBUTION =====
```

```
def example_source_attribution():
    """RAG with source tracking"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: SOURCE ATTRIBUTION")
    print("=" * 70)

    documents = [
        Document(
            page_content="Deep Learning uses neural networks with
multiple layers",
            metadata={"source": "ml_basics.pdf", "page": 1}
        ),
        Document(
            page_content="CNNs are effective for image recognition
tasks",
            metadata={"source": "computer_vision.pdf", "page": 5}
        ),
    ]

    # Setup
    splitter = RecursiveCharacterTextSplitter(chunk_size=100,
chunk_overlap=20)
    chunks = splitter.split_documents(documents)

    embeddings = OpenAIEmbeddings()
    vector_store = FAISS.from_documents(chunks, embeddings)
    retriever = vector_store.as_retriever(search_kwargs={"k": 2})

    # Retrieve with sources
    query = "What is deep learning?"
    results = retriever.invoke(query)

    print(f"Query: {query}\n")
    print("Retrieved documents with sources:")
    for i, doc in enumerate(results, 1):
        print(f"\n  Document {i}:")
```

```
        print(f"    Content: {doc.page_content}")
        print(f"    Source: {doc.metadata.get('source', 'unknown')}")

# ===== EXAMPLE 5: REACT AGENT WITH RAG =====

def example_react_agent_rag():
    """Combine ReAct agent pattern with RAG"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: REACT AGENT WITH RAG")
    print("=" * 70)

    from langchain_core.tools import tool
    from langchain.agents import create_tool_calling_agent,
AgentExecutor
    from langchain_core.prompts import ChatPromptTemplate

    # Create knowledge base
    documents = [
        Document(page_content="Paris is the capital of France.
Population: ~2.2 million"),
        Document(page_content="France is in Western Europe. Currency:
Euro. Language: French"),
        Document(page_content="London is the capital of UK. Population:
~9 million"),
    ]

    # Setup RAG
    splitter = RecursiveCharacterTextSplitter(chunk_size=100,
chunk_overlap=20)
    chunks = splitter.split_documents(documents)

    embeddings = OpenAIEmbeddings()
    vector_store = FAISS.from_documents(chunks, embeddings)
    retriever = vector_store.as_retriever(search_kwargs={"k": 2})

    # Create tool that uses RAG
    @tool
```

```
    def search_knowledge_base(query: str) -> str:
        """Search the knowledge base for information"""
        docs = retriever.invoke(query)
        return "\n".join([doc.page_content for doc in docs])

    # Create agent
    tools = [search_knowledge_base]
    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

    agent = create_tool_calling_agent(
        llm,
        tools,
        ChatPromptTemplate.from_messages([
            ("system", "You are a geography expert. Use the knowledge
base to answer questions."),
            ("human", "{input}")
        ])
    )

    executor = AgentExecutor.from_agent_and_tools(agent=agent,
tools=tools, max_iterations=5)

    # Run
    query = "What is the capital of France and what country is it in?"
    print(f"Query: {query}\n")

    result = executor.invoke({"input": query})
    print(f"Answer: {result['output']}")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_basic_rag()
    example_rag_with_llm()
    example_multi_document_rag()
    example_source_attribution()
    example_react_agent_rag()
```

```
    print("\n" + "=" * 70)
    print("✓ All RAG examples completed")
    print("=" * 70)
```

Memory Management

Theory & Core Concepts

What is Memory in LangChain?

Memory is the mechanism to maintain conversation history and context across multiple interactions with the LLM.

Why Memory Matters

1. **Context Preservation**: LLM remembers previous messages
2. **Coherent Conversations**: Follow-up questions refer to prior context
3. **State Management**: Track conversation state and variables
4. **Token Efficiency**: Buffer windows to manage token costs
5. **Persistent Memory**: Optional storage between sessions

Memory Types

Type	Purpose	Trade-offs
Buffer	Simple list of messages	No summarization; grows unbounded
Summary	Summarize old messages	Loses detail; extra LLM calls
Entity	Track entities (people, places)	Complex; requires extraction
Token Buffer	Keep last N tokens	Intelligent; preserves recent context

---

Line-by-Line Code Breakdown

Simple Message Buffer

```
from langchain_core.messages import HumanMessage, AIMessage,
SystemMessage

# Create memory as list of messages
memory = [
    SystemMessage(content="You are a helpful assistant")
]

# Add messages as conversation continues
def chat(user_input: str):
    # Add user message
    memory.append(HumanMessage(content=user_input))
    # Why: Preserve user input in memory
    # How it helps: LLM can reference previous messages

    # Get response
    response = llm.invoke(memory)

    # Add assistant message
    memory.append(AIMessage(content=response.content))
    # Why: Add assistant response to memory
    # How it helps: Future messages can reference what assistant said

    return response.content

# Conversation flow
chat("What is Python?")  # memory grows
chat("How do I install it?")  # memory has previous question
```

ConversationBufferMemory (LangChain)

```
from langchain.memory import ConversationBufferMemory

# Create buffer memory
memory = ConversationBufferMemory(
    return_messages=True
    # Why: Return as Message objects (needed for LCEL)
)

# Add conversation
memory.chat_memory.add_user_message("Hello")
# Why: Structured way to add messages
# How it helps: Handles formatting automatically

memory.chat_memory.add_ai_message("Hi, how can I help?")

# Retrieve memory
history = memory.load_memory_variables({})
# Returns: {"history": [HumanMessage, AIMessage]}
# Why: Get all messages in conversation
# How it helps: Pass to LLM or save to database
```

Token-Limited Memory

```
from langchain.memory import ConversationTokenBufferMemory

# Keep only recent messages within token limit
memory = ConversationTokenBufferMemory(
    llm=llm,
    max_token_limit=100,  # Keep ~100 tokens of history
    # Why: Prevent token explosion in long conversations
    # How it helps: Manages costs; keeps recent context
)

# Add messages - old ones auto-removed if over limit
memory.chat_memory.add_user_message("Long message...")
memory.chat_memory.add_ai_message("Long response...")

# Memory auto-prunes to fit token limit
```

Complete, Ready-to-Run Code Block

```
"""
Complete Memory Management Example
Demonstrates conversation buffer, token management, and persistence
"""

from langchain_core.messages import HumanMessage, AIMessage,
SystemMessage
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
import os
from dotenv import load_dotenv

load_dotenv()

if not os.environ.get("OPENAI_API_KEY"):
    raise ValueError("OPENAI_API_KEY not set")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# ===== EXAMPLE 1: SIMPLE MESSAGE LIST MEMORY =====

def example_simple_memory():
    """Basic memory using list of messages"""
    print("\n" + "=" * 70)
    print("EXAMPLE 1: SIMPLE MESSAGE LIST MEMORY")
    print("=" * 70)

    # Initialize memory
    memory = [
        SystemMessage(content="You are a helpful assistant about
Python")
    ]

    def chat(user_input: str) -> str:
        """Add user message and get response"""
```

```
        # Add user message
        memory.append(HumanMessage(content=user_input))

        # Get response from LLM
        response = llm.invoke(memory)

        # Add assistant message
        memory.append(AIMessage(content=response.content))

        return response.content

    # Conversation
    print("User: What is Python?")
    response1 = chat("What is Python?")
    print(f"Assistant: {response1[:100]}...\n")

    print("User: How do I install it?")
    response2 = chat("How do I install it?")
    print(f"Assistant: {response2[:100]}...\n")

    print("User: Any tips for beginners?")
    response3 = chat("Any tips for beginners?")
    print(f"Assistant: {response3[:100]}...\n")

    # Show memory
    print(f"Memory size: {len(memory)} messages")
    print("Memory contents:")
    for i, msg in enumerate(memory):
        msg_type = type(msg).__name__
        content_preview = msg.content[:50]
        print(f"  {i}. [{msg_type}] {content_preview}...")

# ===== EXAMPLE 2: CONVERSATION BUFFER MEMORY =====

def example_buffer_memory():
    """LangChain's ConversationBufferMemory"""
    print("\n" + "=" * 70)
```

```
    print("EXAMPLE 2: CONVERSATION BUFFER MEMORY")
    print("=" * 70)

    # Create memory
    memory = ConversationBufferMemory(
        return_messages=True
    )

    # Simulate conversation
    exchanges = [
        ("What is machine learning?", "ML is a subset of AI..."),
        ("Give me an example", "An example is email spam detection..."),
        ("How is it trained?", "Training involves feeding data..."),
    ]

    for user_msg, ai_msg in exchanges:
        memory.chat_memory.add_user_message(user_msg)
        memory.chat_memory.add_ai_message(ai_msg)
        print(f"User: {user_msg}")
        print(f"Assistant: {ai_msg[:60]}...\n")

    # Retrieve all messages
    history = memory.load_memory_variables({})
    print(f"Total messages in memory: {len(history['history'])}")

# ===== EXAMPLE 3: MEMORY WITH PROMPT TEMPLATE =====

def example_memory_with_prompt():
    """Use memory in a prompt template"""
    print("\n" + "=" * 70)
    print("EXAMPLE 3: MEMORY WITH PROMPT TEMPLATE")
    print("=" * 70)

    memory = ConversationBufferMemory(return_messages=True)

    # Create prompt template with memory variable
    prompt = ChatPromptTemplate.from_messages([
```

```
        ("system", "You are a helpful coding assistant"),
        ("placeholder", "{chat_history}"),
        ("human", "{input}")
    ])

    def chat_with_memory(user_input: str):
        # Get memory
        chat_history = memory.load_memory_variables({})["history"]

        # Create messages
        formatted_prompt = prompt.invoke({
            "chat_history": chat_history,
            "input": user_input
        })

        # Get response
        response = llm.invoke(formatted_prompt)

        # Add to memory
        memory.chat_memory.add_user_message(user_input)
        memory.chat_memory.add_ai_message(response.content)

        return response.content

    # Conversation
    queries = [
        "What is a decorator in Python?",
        "Show me an example",
        "Can I stack multiple decorators?"
    ]

    for query in queries:
        print(f"Q: {query}")
        answer = chat_with_memory(query)
        print(f"A: {answer[:80]}...\n")

# ===== EXAMPLE 4: MANUAL CONVERSATION LOOP =====
```

```
def example_manual_conversation():
    """Manual conversation management"""
    print("\n" + "=" * 70)
    print("EXAMPLE 4: MANUAL CONVERSATION LOOP")
    print("=" * 70)

    # Initialize conversation
    messages = [
        SystemMessage(content="You are a travel guide. Answer questions
about destinations")
    ]

    # Conversation turns
    conversation = [
        "Tell me about Paris",
        "What's the best time to visit?",
        "How long should I stay?",
        "What about food there?"
    ]

    for i, user_input in enumerate(conversation, 1):
        # Add user message
        messages.append(HumanMessage(content=user_input))

        # Get response
        response = llm.invoke(messages)

        # Add assistant message
        assistant_message = response.content
        messages.append(AIMessage(content=assistant_message))

        # Display
        print(f"Turn {i}:")
        print(f"  User: {user_input}")
        print(f"  Assistant: {assistant_message[:100]}...")
        print(f"  (Message count: {len(messages)})\n")
```

```
# ===== EXAMPLE 5: SUMMARIZATION FOR MEMORY =====

def example_memory_summarization():
    """Summarize old messages to manage memory size"""
    print("\n" + "=" * 70)
    print("EXAMPLE 5: MEMORY WITH SUMMARIZATION")
    print("=" * 70)

    messages = [
        SystemMessage(content="You are a helpful assistant")
    ]

    # Simulate long conversation
    conversation_turns = [
        ("What is Python?", "Python is a programming language..."),
        ("When was it created?", "Python was created in 1991..."),
        ("Why is it popular?", "Python is popular because..."),
        ("Show me example", "Here is an example..."),
        ("Explain decorators", "Decorators are wrappers..."),
    ]

    for user, assistant in conversation_turns:
        messages.append(HumanMessage(content=user))
        messages.append(AIMessage(content=assistant))

    print(f"Initial message count: {len(messages)}\n")

    # Summarize old messages
    old_messages = messages[1:4]  # Keep system msg and last few

    # Create summary prompt
    summary_prompt = ChatPromptTemplate.from_messages([
        ("system", "Summarize the conversation below:"),
        ("human", "\n".join([f"{m.type}: {m.content}" for m in
old_messages]))
    ])
```

```
    # Get summary
    summary_response = llm.invoke(summary_prompt)
    summary = summary_response.content

    print(f"Original messages (3): {len(old_messages)}")
    print("Content:")
    for msg in old_messages:
        print(f"  - {msg.content[:50]}...")

    print(f"\nSummary:")
    print(f"  {summary[:150]}...\n")

    # Replace old messages with summary
    new_messages = [
        messages[0],  # System
        SystemMessage(content=f"Summary of earlier conversation:
{summary}"),
        messages[-2],  # Last user message
        messages[-1],  # Last assistant message
    ]

    print(f"After summarization: {len(new_messages)} messages (vs
{len(messages)} original)")

# ===== EXAMPLE 6: MEMORY WITH TURN LIMIT =====

def example_turn_limited_memory():
    """Keep only last N conversation turns"""
    print("\n" + "=" * 70)
    print("EXAMPLE 6: TURN-LIMITED MEMORY")
    print("=" * 70)

    max_turns = 3  # Keep only last 3 user/assistant pairs
    messages = [SystemMessage(content="You are a helpful assistant")]

    conversation = [
```

```
        "Hello",
        "What is AI?",
        "Tell me more",
        "How does it work?",
        "Any examples?",
        "What's deep learning?"
    ]

    for i, user_input in enumerate(conversation, 1):
        # Add messages
        messages.append(HumanMessage(content=user_input))
        messages.append(AIMessage(content=f"Response to: {user_input}"))

        # Keep only last N turns (plus system message)
        if len(messages) > max_turns * 2 + 1:
            # Remove 2nd and 3rd messages (oldest user/assistant pair)
            messages = [messages[0]] + messages[3:]

        print(f"Turn {i}: Message count = {len(messages)}")

    print(f"\nFinal memory preserves:")
    print(f"  - System message")
    print(f"  - Last {max_turns} conversation turns ({max_turns * 2}
messages)")

# ===== MAIN EXECUTION =====

if __name__ == "__main__":
    example_simple_memory()
    example_buffer_memory()
    example_memory_with_prompt()
    example_manual_conversation()
    example_memory_summarization()
    example_turn_limited_memory()

    print("\n" + "=" * 70)
```

```
    print("✓ All memory management examples completed")
    print("=" * 70)
```

Complete End-to-End Examples

This section provides complete, production-ready examples combining multiple concepts.

Example 1: Complete RAG + Agent Application

```
"""
Production-Ready RAG + Agent Application
Combines retrieval, reasoning, and multi-step problem solving
"""

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate
from langchain.memory import ConversationBufferMemory
import os
from dotenv import load_dotenv

load_dotenv()

class KnowledgeAssistant:
    """RAG + Agent assistant with memory"""

    def __init__(self, documents_text: list[str]):
        """Initialize with document corpus"""
        # Setup documents
        documents = [Document(page_content=text) for text in
documents_text]

        # Create vector store
        splitter = RecursiveCharacterTextSplitter(chunk_size=200,
chunk_overlap=50)
        chunks = splitter.split_documents(documents)

        embeddings = OpenAIEmbeddings()
        self.vector_store = FAISS.from_documents(chunks, embeddings)
        self.retriever = self.vector_store.as_retriever(search_kwargs=
{"k": 3})
```

```
        # Setup LLM
        self.llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

        # Setup memory
        self.memory = ConversationBufferMemory(return_messages=True)

        # Create search tool
        @tool
        def search_knowledge_base(query: str) -> str:
            """Search the knowledge base"""
            docs = self.retriever.invoke(query)
            return "\n\n".join([d.page_content for d in docs])

        # Create agent
        self.tools = [search_knowledge_base]

        agent = create_tool_calling_agent(
            self.llm,
            self.tools,
            ChatPromptTemplate.from_messages([
                ("system", "You are a helpful assistant. Use the
knowledge base to answer questions."),
                ("human", "{input}")
            ])
        )

        self.executor = AgentExecutor.from_agent_and_tools(
            agent=agent,
            tools=self.tools,
            max_iterations=5
        )

    def query(self, user_input: str) -> str:
        """Process user query with agent"""
        # Add to memory
        self.memory.chat_memory.add_user_message(user_input)
```

```
        # Execute
        result = self.executor.invoke({"input": user_input})

        # Add to memory
        self.memory.chat_memory.add_ai_message(result["output"])

        return result["output"]

# Usage
if __name__ == "__main__":
    documents = [
        "Python is a high-level language created in 1991...",
        "Machine Learning uses algorithms to learn from data...",
        "Neural Networks are inspired by biological neurons...",
    ]

    assistant = KnowledgeAssistant(documents)

    print("Assistant ready. Ask questions about the knowledge base.\n")

    # Simulate conversation
    queries = [
        "What is Python?",
        "When was it created?",
        "Tell me about machine learning"
    ]

    for query in queries:
        print(f"Q: {query}")
        response = assistant.query(query)
        print(f"A: {response}\n")
```

Conclusion

This guide has covered all major LangChain concepts with theory, code breakdowns, complete examples, and data flow explanations. Each topic is production-ready and can be

adapted for your specific use cases.

Key Takeaways

1. **Start with LCEL**: Use `|` operator to compose chains
2. **Leverage Templates**: Reuse prompts across application
3. **Validate with Pydantic**: Ensure structured outputs
4. **Use Tools**: Extend LLM capabilities beyond text
5. **Implement Memory**: Maintain context across turns
6. **Implement RAG**: Ground responses in real data
7. **Build Agents**: Autonomous problem solving

Next Steps

- Implement a RAG application with your documents
- Create custom tools for your domain
- Build a conversational agent with memory
- Deploy to production with proper error handling
- Monitor and iterate based on user feedback

End of Guide